

Machine Learning Fundamentals: From Data to Prediction

Prepared by: Dr. T. Saranya

Assistant Professor – I, IT | VCET, Madurai

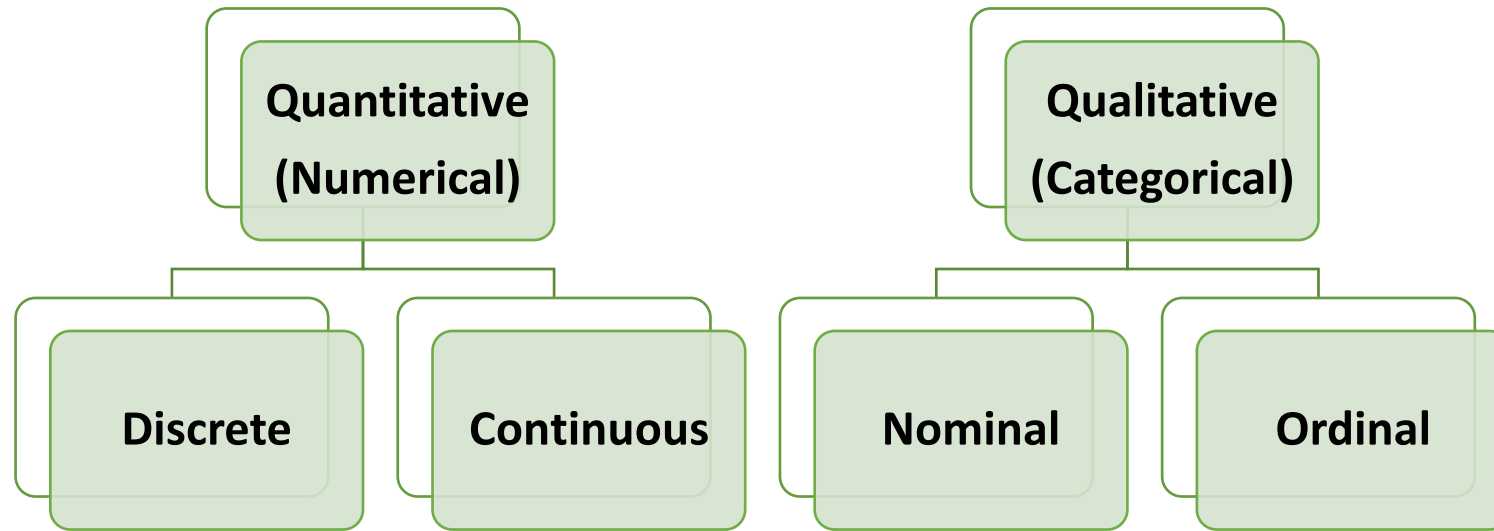
Alumna – TCE | Former Software Developer | Former Research Engineer – Zoho R&D

[Scopus](#) | [Google Scholar](#) | [ORCID](#)

Agenda

- Understanding Data in Machine Learning
- Introduction to Machine Learning
- Types of Machine Learning & ML Tasks
- Machine Learning Process & Data Preprocessing
- Building & Evaluating Machine Learning Models
- Outcome of the Session

Types of Data in ML



Quantitative Data (Numerical)

- Quantitative Data = "How much / How many?"
- It represents **quantity or numerical measurement**.

Examples:

- Age = 25 years
- Salary = ₹50,000
- Temperature = 32.5°C
- Number of students = 60
- Height = 165 cm

Types of Quantitative Data

Type	Meaning	Example
Discrete	Countable	Number of students = 50
Continuous	Measurable	Temperature = 32.5°C

Qualitative Data (Categorical)

- Qualitative Data = "What type / Which category?"
- It describes **qualities, characteristics, or categories** rather than quantities.

Examples:

- Color = Red
- Gender = Female
- Blood group = A+
- Education = Master's
- Satisfaction = Excellent

Types of Qualitative Data

Type	Order?	Example
Nominal	No	Red, Blue, Green
Ordinal	Yes	Poor, Average, Good, Excellent

Understanding

Student	Age	Height	Gender	Education	Passed
A	20	165.5	Female	UG	Yes
B	21	172.3	Male	UG	No
C	22	168.2	Female	PG	Yes

Now classify each column:

Features	Data type	More specific type
Age	Quantitative	Numerical / Discrete
Height	Quantitative	Numerical / Continuous
Gender	Qualitative	Categorical / Nominal
Education	Qualitative	Categorical / Ordinal
Passed	Qualitative	Categorical / Binary



Categorical → Nominal / Ordinal

Binary → Categorical data with exactly two categories

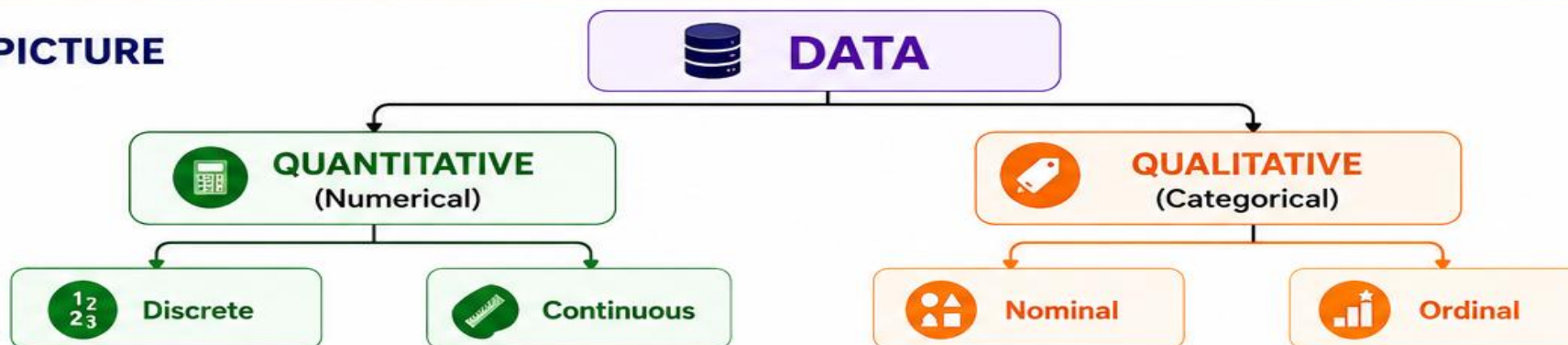
Multiclass → Categorical data with three or more categories



TYPES OF DATA IN MACHINE LEARNING



1 THE BIG PICTURE



2 QUANTITATIVE DATA = "HOW MUCH / HOW MANY?"

It represents quantity or numerical measurement.



Age
= 25 years



Salary
= ₹50,000



Temperature
= 32.5 °C



Number of
students = 60



Height
= 165 cm

It can be divided into:

Type	Meaning	Example
Discrete	Countable	Number of students = 50
Continuous	Measurable	Temperature = 32.5 °C



So: Quantitative → Numerical → Discrete / Continuous

3 QUALITATIVE DATA = "WHAT TYPE / WHICH CATEGORY?"

It describes qualities, characteristics, or categories rather than quantities.



Color
= Red



Gender
= Female



Blood group
= A+



Education
= Master's



Satisfaction
= Excellent

It can be divided into:

Type	Order?	Example
Nominal	✗ No	Red, Blue, Green
Ordinal	✓ Yes	Poor, Average, Good, Excellent



So: Qualitative → Categorical → Nominal / Ordinal

4 VERY SIMPLE EXAMPLE

Information about students:

Student	Age	Height	Gender	Education	Passed
A	20	165.5	Female	UG	Yes
B	21	172.3	Male	UG	No
C	22	168.2	Female	PG	Yes

Classify each column:

Variable	Data type	More specific type
Age	Quantitative	Numerical / Discrete
Height	Quantitative	Numerical / Continuous
Gender	Qualitative	Categorical / Nominal
Education	Qualitative	Categorical / Ordinal
Passed	Qualitative	Categorical / Binary



Why is "Passed" qualitative?

Even though we may represent it as:

Yes = 1 | No = 0

The numbers 1 and 0 are **just labels**.
They don't represent a measurable quantity.



MACHINE
LEARNING

Machine Learning

Humans learn from experience and adapt their actions for future tasks.



Can machines adapt their behavior using experience?



Since the 1950s, researchers have been trying to develop techniques that enable machines to learn. There have been much success in areas such as **automatic control**, **recognition systems** and **natural language processing**. Other successes are emerging.



Automatic
Control



Recognition
Systems

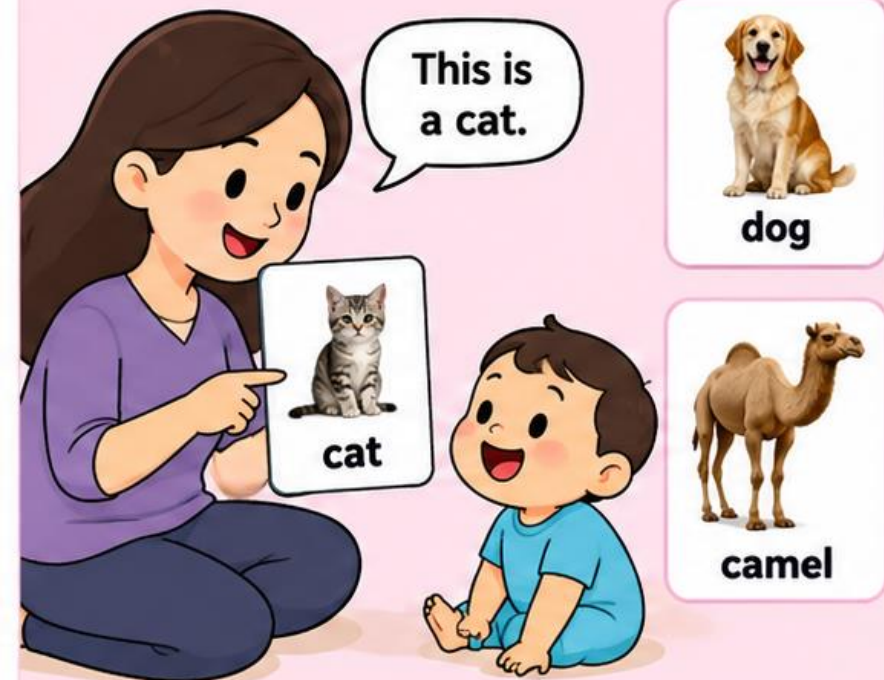


Natural Language
Processing

Machine Learning

Learn from Experience, Just Like Humans!

How we teach a baby?



Later, when the baby sees a different cat (it's not the same cat as before)...

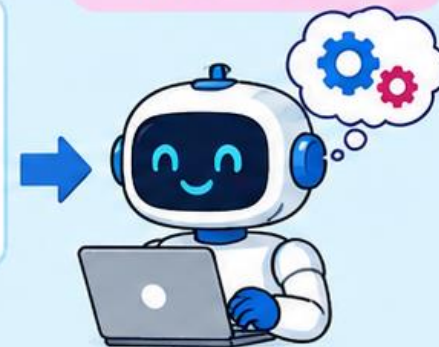


Similarly, in Machine Learning...

1. We give examples (training data)



2. The machine learns patterns from the examples



3. When it sees new data (unseen before)...



4. It can make a prediction or decision.

This is a cat!



In simple words:

Machine Learning helps computers **learn from data** and **improve their performance** without being explicitly programmed for every task.

Learn
Adapt
Improve



What is Machine Learning?

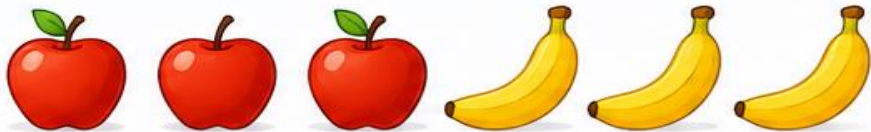


*“Machine Learning is the ability of a computer to **LEARN** from **DATA**, **DISCOVER PATTERNS**, and **MAKE PREDICTIONS** — without being explicitly programmed for every task.”*



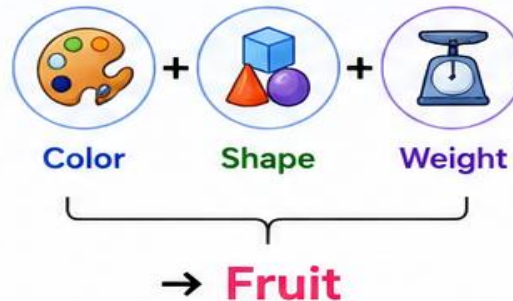
A Simple Example

Show the computer many examples of fruits:



Apple → Apple → Apple Banana → Banana → Banana

The ML model
learns patterns such as:



Then...



★ Key Message

Machine Learning is **not** about programming every rule.
It is about enabling machines to **LEARN** the rules from **DATA**.

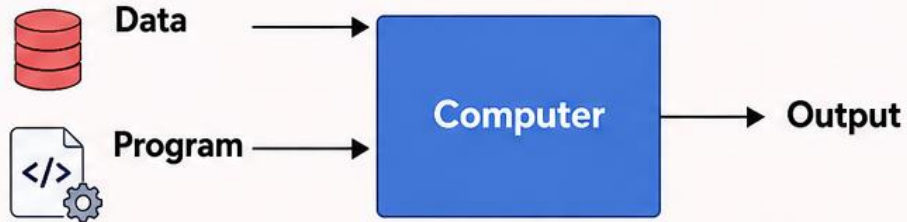




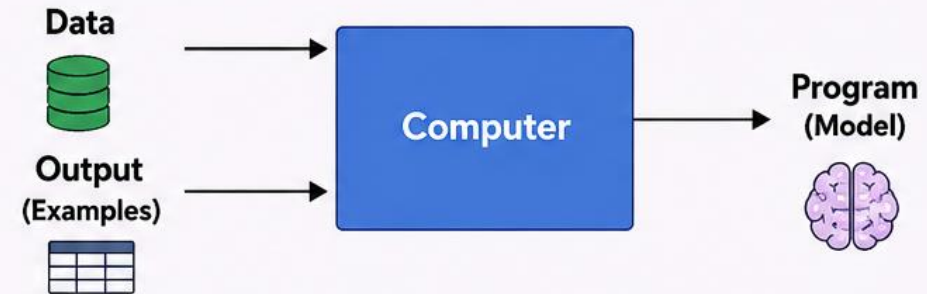
Traditional Programming vs Machine Learning



Traditional Programming



Machine Learning



We give **Rules** + **Data**

VS



We give **Data** + **Examples**



Computer **follows our rules**

VS



Computer **learns patterns**



Rules are **explicitly programmed**

VS



Rules are **learned from data**



Output is **generated**

VS



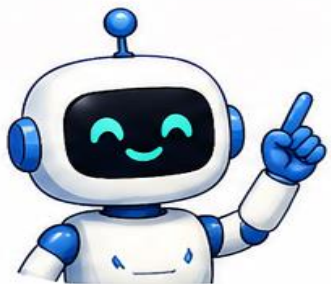
Model **predicts** new outcomes



Key Takeaway

Traditional Programming tells the computer **WHAT TO DO**.
Machine Learning teaches the computer **HOW TO DO IT**.





How Does a Machine Actually Learn?



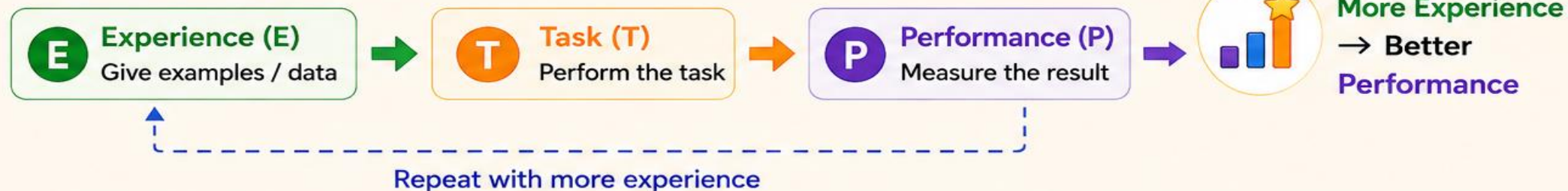
A machine learns when its **performance improves** through **experience**.

The 3 Important Terms

	E – Experience	T – Task	P – Performance
 Meaning	What does the machine learn from?	What does the machine have to do?	How do we know it is getting better?
 Simple Example	Many labeled pictures of cats and dogs	Identify whether a new picture is a cat or a dog	Percentage of correct predictions on new pictures (accuracy)
 Another Example	Past house data (size, location, price...)	Predict the price of a new house	How close the predicted price is to the actual price (mean error)



The ML Learning Idea



Basic ML Terminology



Let's understand **4 fundamental terms** using a simple example dataset.

Dataset

A collection of samples or records.



Feature (X)

Input variables or attributes used by the model.



Sample / Instance

One individual observation or record.



Target (y)

The output or value we want the model to predict.



Example Dataset

Sample / Instance	Features (X)			Target (y)
	Study Hours (per day)	Previous Score (%)	Attendance (%)	Pass Exam (Yes / No)
1	2	60	75	No
2	4	70	80	Yes
3	3	65	60	No
4	6	85	90	Yes
5	5	80	85	Yes

Each row is a **Sample / Instance** (one student record).

This column is the **Target (y)** – what we want to predict.

These columns are **Features (X)** – inputs used by the model.



Here, the model will use Study Hours, Previous Score, and Attendance (**Features**) to predict whether the student will **Pass the Exam (Target)**.

Dataset, Features (X), Sample / Instance and Target (Y)



A machine learning dataset is made of **examples (rows)**, input variables (**features**) and the output we want to predict (**target**).

Dataset

A collection of samples (rows).

Sample / Instance

One individual record (row).

Sample / Instance	Features (X)			Target (Y) – Output
	Height (cm)	Weight (kg)	Age (years)	Admission (Yes / No)
1	160	55	20	Yes
2	170	65	22	No
3	155	50	19	Yes
4	165	60	21	No

← Target (Y)

The output we want to predict.

Quick Summary

- ✓ Each row → Sample / Instance
- ✓ Columns in green → Features (X)
- ✓ Column in red → Target (Y)

Features (X)

Height, Weight, Age
(Inputs)



Target (Y)

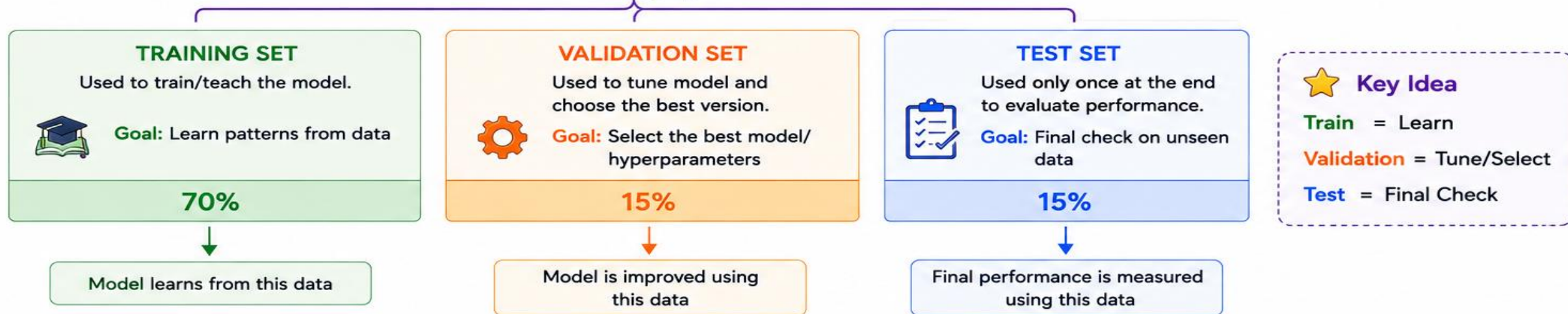
Admission
(Yes / No)

Dataset Splitting in Machine Learning



We split the dataset into **Train**, **Validation** and **Test** sets so that the model can learn, improve and finally be evaluated on unseen data.

Three-Way Split



Example

Suppose we have a dataset with **100 samples** (rows).

1	2	3	4	5	5	6	7	8	9	10		70	71	72	73	74		85	86	87	88	89		100
---	---	---	---	---	---	---	---	---	---	----	-------	----	----	----	----	----	-------	----	----	----	----	----	-------	-----

70 Training Set (70%)

- 70 samples
- Used to train the model



15 Validation Set (15%)

- 15 samples
- Used to tune hyperparameters and select the best model



15 Test Set (15%)

- 15 samples
- Used only once at the end to evaluate the final model



Common Split Ratios

• **70% – 15% – 15%** (Train – Validation – Test)

• **80% – 10% – 10%**

• **80% – 20%** (Train – Test only)



The exact ratio depends on the dataset size and the problem.

Train – Test Split (**Two-Way Split**)

A simple and widely used way to evaluate a machine learning model

Often
Good Enough
for Many Problems!



We split the dataset into two parts: **Training set** and **Test set**.

The model learns from the training data and is evaluated on the test data (unseen data).



Full Dataset

(e.g., 100 samples)

Split



Training Set

80%

(e.g., 80 samples)



Test Set

20%

(e.g., 20 samples)

1

TRAINING SET – Learn

- Around **80%** of the data.
- Used to train (build) the model.
- The model learns patterns from this data.
- The model has access to this data during training.



Model learns
from training data

2

TEST SET – Final Check

- Around **20%** of the data.
- Not used during training (unseen data).
- Used only after training is complete.
- Tells us how well the model performs on new, unseen data.



Evaluate the model
on unseen data



Key Takeaway

Train → **Learn**

Test → **Evaluate**

A train–test split is **often enough** for many machine learning problems, especially when the model is simple and no extensive tuning is required.

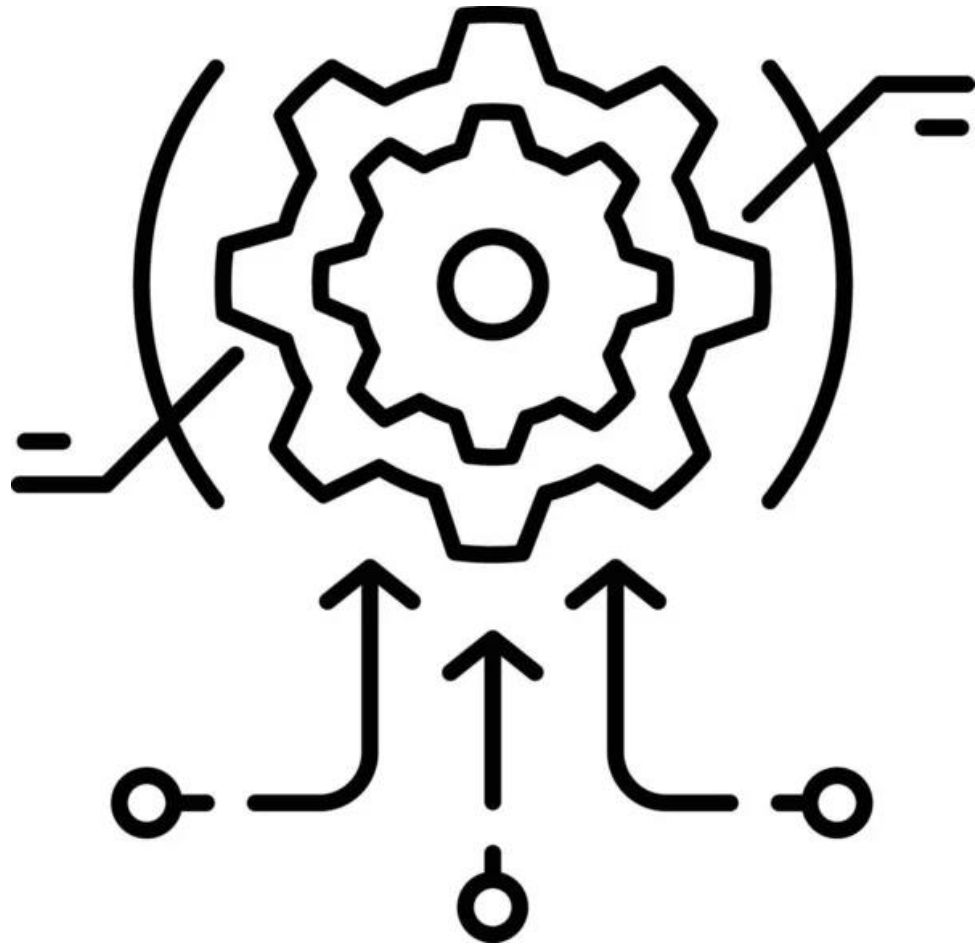


Note

We introduce a separate **validation set** when we need to:

- Tune model settings (hyperparameters)
- Compare different model versions
- Select the best model before final testing

Types of Learning in Machine Learning



TYPES OF MACHINE LEARNING

Machine Learning can be broadly categorized into the following types:

01



**SUPERVISED
LEARNING**

02



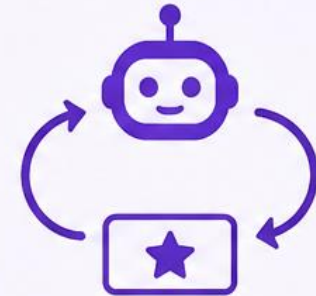
**UNSUPERVISED
LEARNING**

03



**SEMI-SUPERVISED
LEARNING**

04



**REINFORCEMENT
LEARNING**



These types differ in the kind of data available and the way the model learns from it.

SUPERVISED LEARNING



Supervised Learning is a type of Machine Learning where the model learns from **labeled data** (features **X** and target **y**) to make predictions on new, unseen data.

• HOW IT WORKS? •

1 Labeled Dataset

We have data with input features (**X**) and correct output (target **y**).

Features (X)					Target (y)
X ₁	X ₂	X ₃	...	X _n	y
2.3	1.1	3.0	...	0.2	0
1.5	3.2	0.8	...	1.0	1
3.1	0.5	1.2	...	0.3	0
...	...	...	...	...	...
2.0	1.7	2.1	...	0.9	1

Model learns the relationship between features (**X**) and target (**y**).

2 Model Learning

The model takes both **X** and **y** and learns patterns/relationship.



Learn mapping:
 $f(X) \rightarrow y$

3 Make Predictions

For new data, we provide only features (**X**) to the trained model.

New Input (Features X)

2.6	1.3	2.7	...	0.4
-----	-----	-----	-----	-----



Predicted Output (Target y)
1

4 Get Output

The model outputs the predicted target (**y**) for the new input.



Prediction
1

EXAMPLES



Disease Diagnosis
(Features: Symptoms, Test results → Target: Disease / No Disease)



House Price Prediction
(Features: Area, Location, Bedrooms → Target: Price)



Spam Detection
(Features: Words, Length, Sender info → Target: Spam / Not Spam)



Customer Churn
(Features: Usage, Plan, Support calls → Target: Churn / Not Churn)

KEY POINTS

- ✓ Uses **labeled data** (features **X** and target **y**).
- ✓ Learns a mapping/function $f(X) \rightarrow y$.
- ✓ Used for tasks like **Classification** and **Regression**.
- ✓ **Goal**: Accurate prediction on new, unseen data.



In simple words: We show the model many examples with correct answers (**X** and **y**), so it learns the pattern and can give the answer for new data (only **X**).

UNSUPERVISED LEARNING



Unsupervised Learning is a type of Machine Learning where the model learns from **UNLABELED data** (only **features X**) to discover patterns, structure or groups in the data. There is **NO target (y)** provided.

HOW IT WORKS?

1 Unlabeled Dataset

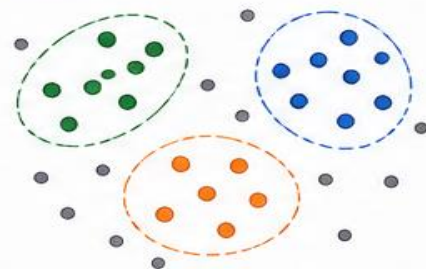
We have data with input features (**X**) but no target (**y**).

X_1	X_2	X_3	...	X_n
2.3	1.1	3.0	...	0.2
1.5	3.2	0.8	...	1.0
3.1	0.5	1.2	...	0.3
...	...	...	...	...
2.0	1.7	2.1	...	0.9

Only features (**X**) are given.
No correct answer / target is provided.

2 Model Learning

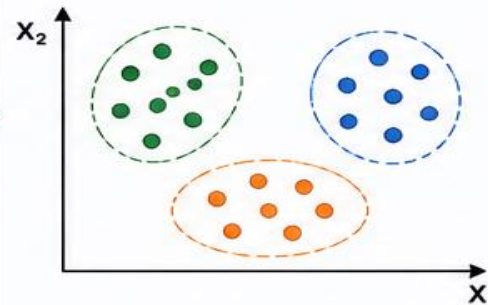
The model explores the data and finds patterns, structure or groups on its own.



Model discovers hidden structure or similarity without any target.

3 Cluster / Pattern Discovery

The model groups similar data points together.



Groups / clusters are formed based on similarity.

4 Get Insights

We get useful insights, patterns or segments from the data.



These insights can be used for decision making, visualization, or further analysis.



EXAMPLES (Common Use Cases)



Customer Segmentation
Group customers with similar behavior.



Market Basket Analysis
Find products that are often bought together.



Image Clustering
Group similar images together.



Anomaly Detection
Find unusual or abnormal patterns.



KEY POINTS

- ✓ Uses **UNLABELED** data (only features **X**).
- ✓ No target (**y**) is provided.
- ✓ Goal is to discover patterns, structure or groups in the data.
- ✓ Common tasks: **Clustering**, **Association**, **Dimensionality Reduction**, **Anomaly Detection**.
- ✓ Useful for exploration, understanding and segmentation.



In simple words: We give the model data without answers, and it automatically finds patterns or groups by itself.

Unsupervised Learning: Discovering Patterns in Data

We give the model data without a target, and it finds patterns on its own!



1 Customer Data (No Target)

We give the model information about 100 customers, but we do not tell it which group they belong to.

Customer ID	Age	Annual Income (USD)	Spending Score (1-100)
1	25	40,000	80
2	32	55,000	77
3	45	60,000	50
4	52	48,000	42
5	60	35,000	20
...	...	...	...

Only input features are given:

- Age, Annual Income, Spending Score (No target/label)

2 Unsupervised Learning

The algorithm looks at the data and finds similar customers based on their characteristics.



Finds patterns automatically!

3 Discovered Groups (Clusters)

The model may discover groups like these:



Group 1

Young + High Spending

- Younger customers
- High income (or moderate)
- High spending score



Group 2

Middle-aged + Medium Spending

- Middle-aged customers
- Medium income
- Medium spending score

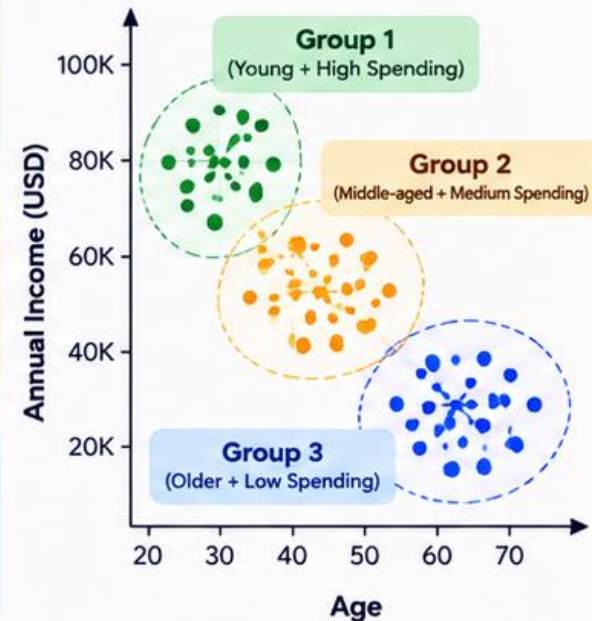


Group 3

Older + Low Spending

- Older customers
- Lower income
- Low spending score

Customer Groups (Example)



What kind of task is this?

Clustering!

Clustering = Grouping similar data points together.



Key Takeaways

- Unsupervised learning uses data without a target/label.
- The model discovers patterns or structure on its own.
- Clustering is one type of unsupervised learning.
- Useful for understanding and segmenting data (e.g., customer groups).

SEMI-SUPERVISED LEARNING



Semi-Supervised Learning is a type of Machine Learning where the model learns from a small amount of **labeled data** (features X and target y) and a large amount of **unlabeled data** (features X only) to make better predictions.

• HOW IT WORKS? •

1 Mixed Dataset

We have a small labeled dataset (X and y) and a large unlabeled dataset (X only).

Labeled Data (small)

X_1	X_2	...	X_n	y
2.3	1.1	...	0.2	0
1.5	3.2	...	1.0	1
3.1	0.5	...	0.3	0
...	...	...	...	...
2.0	1.7	...	0.9	1

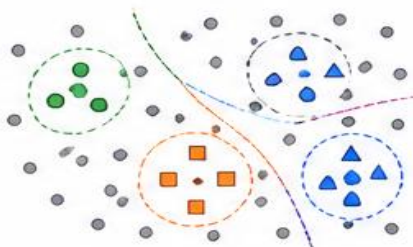
Unlabeled Data (large)

X_1	X_2	...	X_n
0.5	2.1	...	0.7
2.2	1.4	...	1.1
3.0	0.8	...	0.4
...	...	...	...
1.8	2.6	...	0.3

Uses both labeled (X, y) and unlabeled (X) data.

2 Model Learning

The model learns from labeled data and leverages unlabeled data to understand the underlying structure of the data.



- Labeled Class 1
- Labeled Class 2
- ▲ Labeled Class 3
- Unlabeled Data

Unlabeled data helps the model learn better decision boundaries.

3 Make Predictions

For new data, we provide only features (X) to the trained model.

New Input (features X only)

1.9	0.9	...	0.6
-----	-----	-----	-----

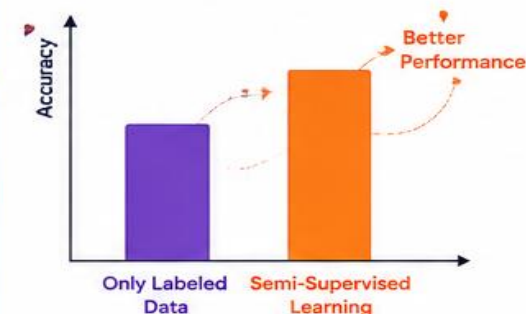


Predicted Output (target y)

1

4 Get Better Predictions

The model uses more data (both labeled and unlabeled) to improve accuracy compared to using only the small labeled data.



More data → More information → Better generalization.



EXAMPLES (Common Use Cases)



Email Spam Detection

Few labeled emails (spam / not spam) + many unlabeled emails



Medical Image Analysis

Few labeled images + many unlabeled images to detect disease



Text Classification

Few labeled documents + many unlabeled documents (reviews, news, etc.)



Customer Feedback Analysis

Few labeled feedbacks + many unlabeled feedbacks to understand sentiment



KEY POINTS

- ✓ Uses both **labeled data** (X, y) and **unlabeled data** (X only).
- ✓ Targets are available only for a small portion of the data.
- ✓ Unlabeled data provides additional information about the data distribution.
- ✓ Goal is to improve model performance when labeled data is limited.
- ✓ Common tasks: **Classification**, **Regression**.
- ✓ Often used when labeling data is expensive or time consuming.



In simple words: We give the model a small number of examples with answers and a large number of examples without answers. The model uses both to learn better and predict more accurately.



Semi-Supervised Learning: Learning with Few Labels

We give the model a small amount of labeled data and a large amount of unlabeled data. The model uses both to learn and make better predictions.

Less Labeling
More Learning!

1 Our Data (Example: Customers)

We have information about 100 customers. Only a few customers have group labels, and most are unlabeled.

Customer ID	Age	Annual Income (USD)	Spending Score (1–100)	Group Label (Available?)
1	25	40,000	80	A (Labeled)
2	32	55,000	77	B (Labeled)
3	45	60,000	50	? (Unlabeled)
4	52	48,000	42	? (Unlabeled)
5	60	35,000	20	? (Unlabeled)
...	...	...	...	...

 **Labeled data**
(small amount)

 **Unlabeled data**
(large amount)

2 How Semi-Supervised Learning Works



Step 1: Use labeled data

The model first learns from the small amount of labeled data.



Step 2: Use unlabeled data

The model also looks at the large amount of unlabeled data to find patterns and structure in the data.

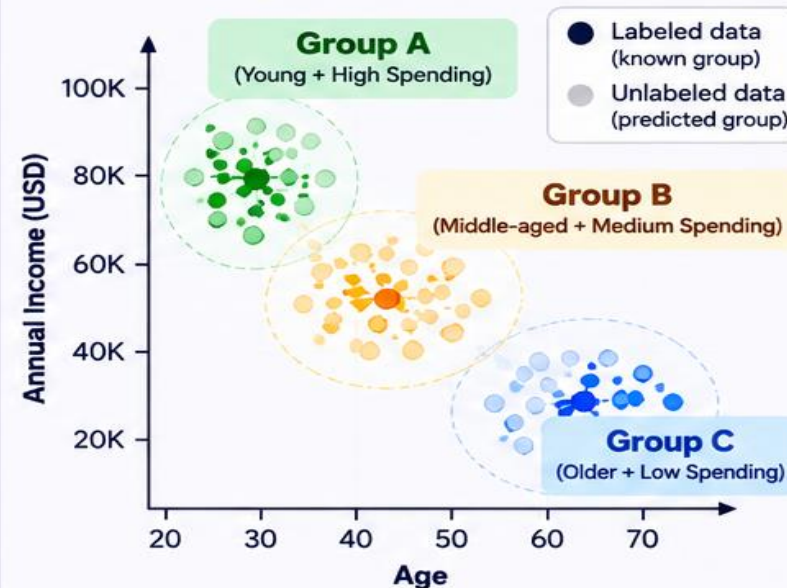


Step 3: Learn better

By using both labeled and unlabeled data, the model can learn more accurately than using only labeled data.

3 Result: Discovering Groups

The model can predict group labels for the unlabeled customers using the patterns it learned.



What kind of task is this?

It is a **Semi-Supervised Learning** task.

Goal:

- ✓ Learn from a small amount of labeled data and a large amount of unlabeled data.
- ✓ Predict the labels for new (unlabeled) data.

When is this useful?



- ✓ When labeling data is expensive or time-consuming.
- ✓ When we have a lot of unlabeled data but only a few labeled examples.
- ✓ Common in real-world problems (e.g., images, text, customer data).



Key Takeaways

- ✓ Uses both labeled and unlabeled data.
- ✓ Learns better than using only labeled data.
- ✓ Useful when labels are limited or costly.
- ✓ Helps the model discover patterns and make more accurate predictions.
- ✓ Can be used for classification or regression tasks.



REINFORCEMENT LEARNING

Reinforcement Learning is a type of Machine Learning where an **agent** learns by interacting with an **environment**. It takes **actions**, receives **rewards** (or **penalties**), and learns a **policy** to maximize cumulative reward.



HOW IT WORKS?



EXAMPLES (Common Use Cases)



Game Playing

Learn to play games (Atari, Chess, Go) and maximize score.



Robotics Control

Control robot movements to perform tasks efficiently.



Autonomous Driving

Learn driving policies to reach destination safely.



Resource Management

Optimize allocation of resources (traffic, cloud, energy, etc.).

KEY POINTS

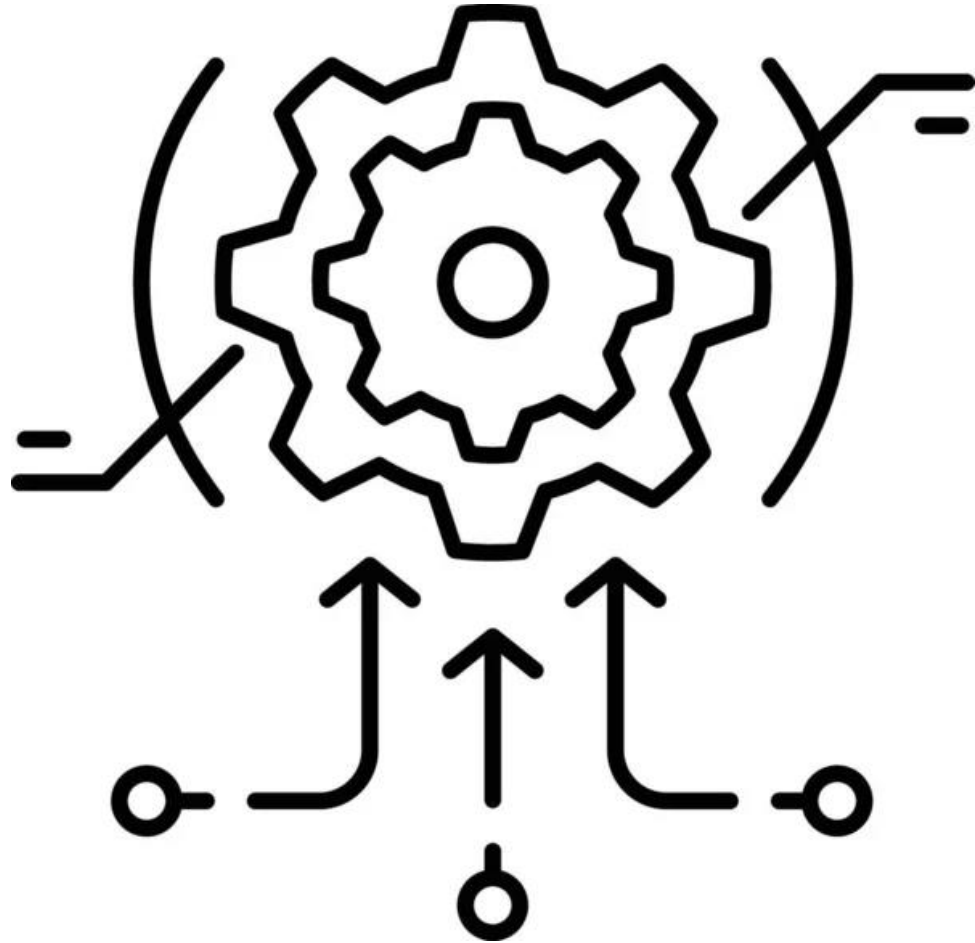
- ✓ Learns by trial and error through interaction.
- ✓ **No labeled** input-output pairs are given.
- ✓ Receives **reward** (positive) or **penalty** (negative).
- ✓ Goal: Learn a **policy** that **maximizes cumulative reward**.
- ✓ Key components: **State** (S), **Action** (A), **Reward** (R), **Policy** (π).
- ✓ Common algorithms: **Q-Learning**, **Deep Q-Network (DQN)**, **Policy Gradient**, **Actor-Critic**, etc.



In simple words: The agent interacts with the environment, takes actions, gets rewards or penalties, and learns what actions to take in different situations to get the maximum reward.



Machine Learning Task



MACHINE LEARNING TASKS



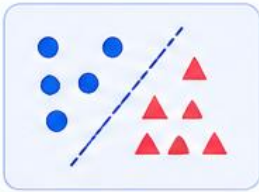
Learning Type = **HOW**
the machine learns



ML Task = **WHAT**
the machine is asked to do

COMMON ML TASKS

1



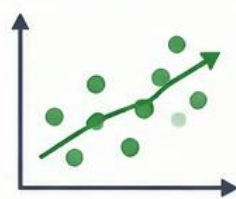
Classification

Predict a category
or class label.

Examples:

Spam / Not Spam,
Cat / Dog, Disease /
No Disease

2



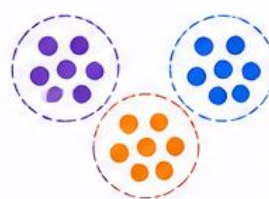
Regression

Predict a continuous
or numerical value.

Examples:

House Price,
Temperature,
Sales Amount

3



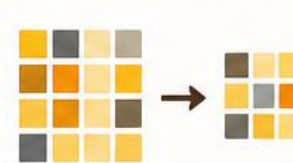
Clustering

Group similar data
points together.

Examples:

Customer Segmentation,
Document Grouping,
Market Analysis

4



Dimensionality Reduction

Reduce the number of
features while retaining
important information.

Examples:

PCA, Feature Selection,
Data Compression,
Visualization

5



Anomaly Detection

Identify unusual or
abnormal observations.

Examples:

Fraud Detection,
Network Intrusion,
Machine Failure



In short: Learning type tells us **HOW** the model learns, and ML tasks tell us **WHAT** the model is expected to do.



CLASSIFICATION TASK



Classification is a machine learning task where we predict a **CATEGORY** or **CLASS LABEL**. It uses **CATEGORICAL (QUALITATIVE)** data as the target / output.

When the target variable is **CATEGORICAL**, we use **CLASSIFICATION** task.

WHY CLASSIFICATION?



We use classification when we need the model to predict one of the predefined classes or categories.

DATA TYPE vs ML TASK

Target / Output Data Type	Example	ML Task
Categorical (qualitative) 	• Yes / No • Spam / Not Spam • Cat / Dog • Disease / No Disease • Male / Female	CLASSIFICATION (Predict a category or class label) 

EXAMPLES OF CLASSIFICATION



Email Spam Detection

Classify email as Spam or Not Spam.

Spam

Not Spam



Disease Prediction

Classify patient as Disease or No Disease.

Disease

No Disease



Image Classification

Classify image as Cat or Dog.

Cat

Dog



Customer Churn

Classify customer as Churn or Not Churn.

Churn

Not Churn

HOW DO WE MAP A REAL PROBLEM TO ML?

1

STEP 1

What is the **PROBLEM**?



Understand the real-world decision or task that needs to be solved.

2

STEP 2

What is the **TASK**?



Define what the model should predict.
(Here: a category / class)

3

STEP 3

What type of **DATA** do we have?



If the target is **CATEGORICAL**, we choose **CLASSIFICATION** task and collect labeled data.



Takeaway: **CATEGORICAL** target → **CLASSIFICATION** task. Classification predicts a category or class label.



Example: Student Pass Prediction

Let's see a simple dataset and identify the target, data type and ML task!

From Data
to Decision!

1 A Small Dataset

Study Hours	Attendance (%)	Assignment Score	Result (Target)
2	60	45	Fail
4	75	60	Pass
6	80	70	Pass
1	50	35	Fail
5	90	85	Pass

2 What is the target?

Result (Pass / Fail)

It is a **categorical** variable because the values are categories (Pass or Fail).

Pass

Fail

3 Do we have a target?

Yes! ✓

Since we have the correct Result (Pass/Fail) in the training data, this is a **Supervised Learning** problem.



We learn from labeled data.

4 What should the model predict?

Pass or Fail

We are predicting a category (class label), so the ML task is **Classification**.



Predict the category!

5 Summary



Target variable:
Categorical
(Pass / Fail)



Data with target:
Supervised Learning



Task:
Classification
(Predict a category)



Key Takeaway

If the target is **CATEGORICAL** and we have **labeled data**, then it is a **SUPERVISED LEARNING** task and the ML task is **CLASSIFICATION**.



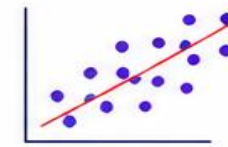
Now I get it!
We are predicting
Pass or Fail!

"Categorical target + Labeled data = Supervised Learning = Classification"



REGRESSION

Regression is a type of Machine Learning where the goal is to predict a **continuous numerical value**. It learns the relationship between input features and a target that is a real number.



INPUT FEATURES
(X)

REGRESSION
MODEL

NUMERICAL
OUTPUT
(y)

HOW IT WORKS?

1 Input Features

We provide input features (X) to the model.

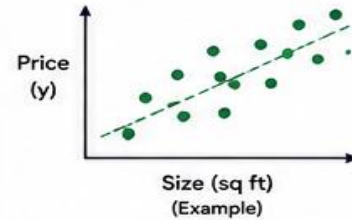
Size (sq ft)	Bedrooms	Location Score
1500	3	8
2000	4	7
1200	2	6
1800	3	9
...	...	...

Example:

House data with features like size, bedrooms, location, etc.

2 Model Learns Relationship

The model learns the relationship between features (X) and target (y) from data.



Example:

Learns how size, bedrooms and location affect the price.

3 Predicts Continuous Value

Given new input, the model predicts a numerical value ($\hat{y}$).



Example:

For a house of 1600 sq ft, 3 bedrooms, location score 8
Predicted Price ($\hat{y}$) = ₹ 75,00,000

4 Minimizes Error

The model adjusts to reduce the difference between predicted value and actual value.

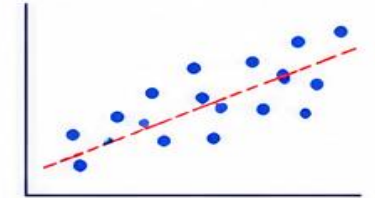


Goal:

Minimize error (loss)
e.g., Mean Squared Error (MSE), MAE, etc.

5 Better Predictions

The model improves and makes more accurate predictions on new data.



Result:

Model generalizes well and predicts close to actual values.

EXAMPLES (Common Use Cases)



House Price Prediction

Predict the price of a house based on size, location, number of rooms, etc.



Sales Forecasting

Predict future sales based on advertising spend, seasonality, etc.



Temperature Prediction

Predict tomorrow's temperature based on weather conditions.



Fuel Consumption

Predict fuel efficiency based on car features, driving conditions, etc.

KEY POINTS

- ✓ Predicts a continuous numerical value.
- ✓ Used when the target variable is quantitative (real number).
- ✓ Finds the best-fitting line/curve that represents the relationship.
- ✓ Performance measured using regression metrics: MSE, MAE, RMSE, R^2 , etc.
- ✓ Common algorithms: Linear Regression, Polynomial Regression, Ridge, Lasso, SVR, Decision Tree Regressor, Random Forest Regressor, etc.
- ✓ Applications: Finance, Real Estate, Sales, Weather, Healthcare, Engineering, etc.



In simple words: Regression finds the relationship between inputs and a continuous number, and predicts the best possible numerical value.

INPUT
FEATURES
(X)

MODEL

PREDICTED
VALUE ($\hat{y}$)
(Continuous Number)



Example: House Price Prediction (Regression)

Let's see a simple dataset and identify the target, data type and ML task!

Predicting a Number!

1 A Small Dataset

Size (sq ft)	Bedrooms	Location Score	Price (Target)
1500	3	8	75
2000	4	7	90
1200	2	6	55
1800	3	9	88
2500	4	8	110



This dataset contains house details and the actual price (in lakhs).

2 What is the target?

Price (in lakhs)

It is a **numerical (continuous)** variable because the value can be any number (e.g., 75, 90, 55, ...).

Target Type:
Continuous (Numerical)

3 Do we have a target?

Yes!

Since we have the actual Price in the training data, this is a **Supervised Learning** problem.



We learn from labeled data.

4 What should the model predict?

House Price (a number)

We are predicting a continuous numerical value, so the ML task is **Regression**.



Predict the number!

5 Visualization



The model learns the best line that fits the data and predicts the price for new houses.

6 Summary



Target variable:
Numerical (Continuous)
(e.g., price)



Data with target:
Supervised Learning



Task:
Regression
(Predict a number)



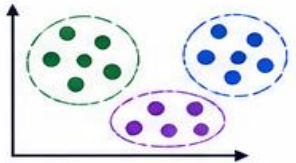
Key Takeaways

- ✓ Regression is used when the target is a continuous numerical value.
- ✓ We have labeled data, so it is a supervised learning task.
- ✓ The goal is to predict a number (not a category).
- ✓ Example applications: house price prediction, sales forecasting, temperature prediction, etc.



In simple words: Regression learns the relationship between input features and a continuous number, and predicts the best possible numerical value.

Past data → Learn the pattern
→ Predict a number!



CLUSTERING

— Unsupervised Machine Learning Task —

INPUT DATA
(No Labels)

CLUSTERING
ALGORITHM

GROUPS /
CLUSTERS
(Output)



Clustering is a machine learning task where we group similar data points together based on their patterns or features, **without using any predefined labels**.

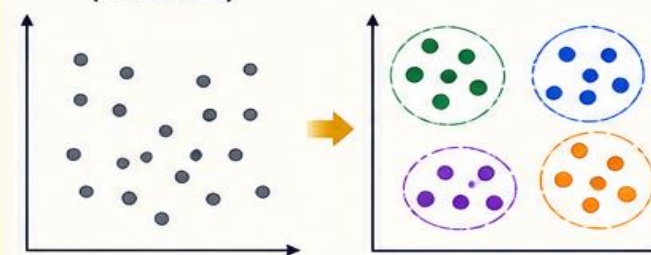
HOW DO WE MAP A REAL PROBLEM TO ML?

- STEP 1**
What is the PROBLEM?
 Understand the real-world problem where we want to find natural groups or segments.
Example:
A retail store wants to group customers with similar purchasing behavior.
- STEP 2**
What is the TASK?
 Define the task as grouping similar data points together into clusters.
Example:
Group customers into clusters such that customers in the same cluster are similar to each other and different from those in other clusters.
- STEP 3**
What type of DATA do we have?
 We have **UNLABELED** data with features only (no target/label).
Example:
Customer data with features like Age, Annual Income, Spending Score, Visit Frequency, etc.

HOW CLUSTERING WORKS?

Input Data
(No Labels)

Output: Clusters



Goal: Maximize similarity **within** a cluster and maximize dissimilarity **between** clusters.

EXAMPLES OF CLUSTERING (REAL WORLD)



Customer Segmentation
Group customers based on purchasing behavior to target marketing.



Product Grouping
Group similar products based on features or attributes.



Location Analysis
Group areas based on crime rate, population, or demographics.



Document Clustering
Group news articles or documents with similar topics.



Gene Clustering
Group genes with similar expression patterns in bioinformatics.

KEY POINTS

- ✓ It is an **UNSUPERVISED** learning task (no labels are given).
- ✓ The goal is to discover natural groups or clusters in the data.
- ✓ Useful for exploration, pattern discovery, and segmentation.
- ✓ **Common Algorithms:** K-Means, Hierarchical Clustering, DBSCAN, Mean Shift, Gaussian Mixture Models, etc.



IN SIMPLE WORDS: Clustering organizes data into groups so that items in the same group are similar, and items in different groups are different.

DATA
(No Labels)

FIND SIMILAR
GROUPS

CLUSTERS
(Output)



Example: Customer Segmentation (Clustering)

Let's see a simple dataset and how the algorithm groups similar customers!

Finding Groups in Data!

1 A Small Dataset (No Labels)

Customer ID	Age (years)	Annual Income (USD)	Spending Score (1-100)
1	23	15,000	39
2	45	85,000	81
3	31	40,000	77
4	22	16,000	40
5	52	88,000	76
6	36	52,000	54
7	58	95,000	12
8	27	30,000	49
...	...	...	...



We only have customer information (no target or labels).

2 Apply Clustering Algorithm

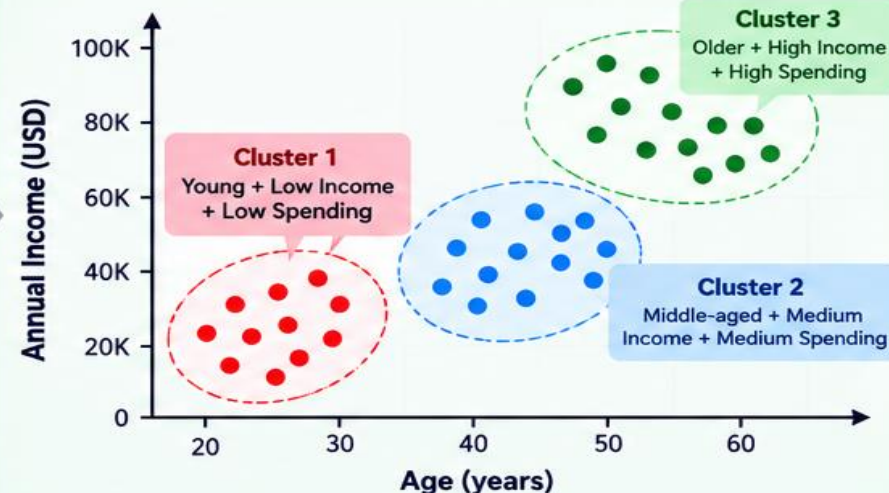
The algorithm looks at the features (Age, Annual Income, Spending Score) and groups customers based on similarity.



Clustering Algorithm
(e.g., K-Means)

3 Discovered Clusters (No Predefined Labels)

The algorithm groups similar customers into clusters.



4 Cluster Profiles (Example Interpretation)

Cluster 1 (Young + Low Spending)



- Younger customers
- Lower income
- Lower spending score

May be students or early career customers

Cluster 2 (Middle-aged + Medium Spending)



- Middle-aged customers
- Medium income
- Medium spending score

May be working professionals

Cluster 3 (Older + High Spending)



- Older customers
- Higher income
- Higher spending score

May be well-settled or premium customers

5 What Have We Learned?

- ✓ We did not give any group labels beforehand.
- ✓ The algorithm automatically found groups based on similarity.
- ✓ This is an Unsupervised Learning task.
- ✓ The task is Clustering.
- ✓ Useful for customer segmentation, market analysis, and pattern discovery.



Key Takeaway

Clustering = Grouping similar data points together without using any predefined labels.



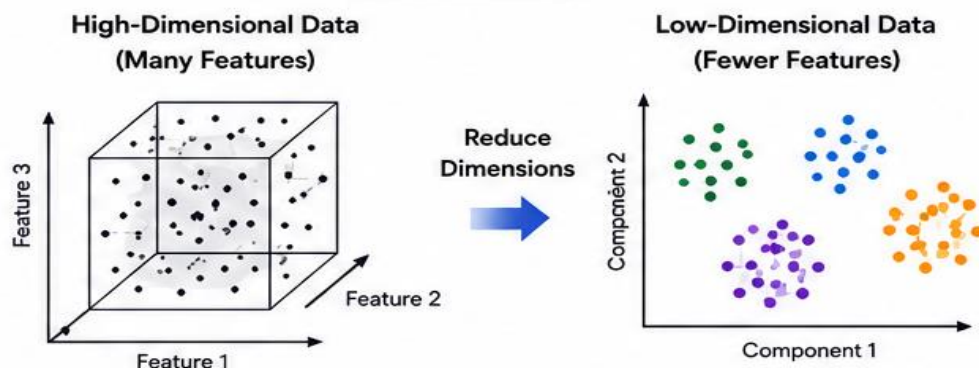
DIMENSIONALITY REDUCTION

Dimensionality Reduction **reduces** the number of input features while keeping the **most important information**.

WHY DO WE USE IT?

- ✓ Simplifies data and makes it easier to understand.
- ✓ Reduces storage and computation time.
- ✓ Improves model performance.

EXAMPLE



In simple words: It keeps the important information and removes the extra or less useful features.

Common Techniques: PCA, t-SNE, LDA, Autoencoders



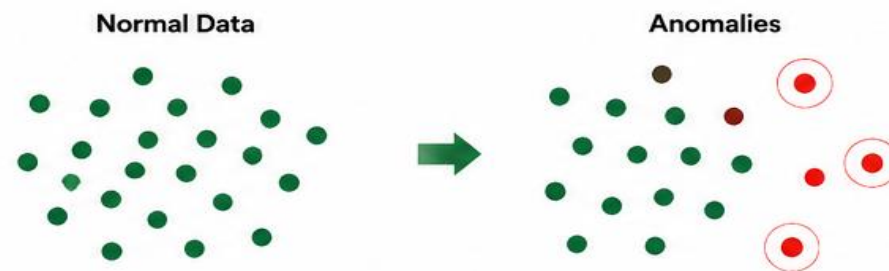
ANOMALY DETECTION

Anomaly Detection finds **rare, unusual or suspicious** data points that are different from the normal pattern.

WHY DO WE USE IT?

- ✓ Finds outliers and unusual events.
- ✓ Helps in fraud detection and error prevention.
- ✓ Useful when labels are not available.

EXAMPLE



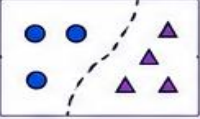
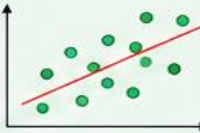
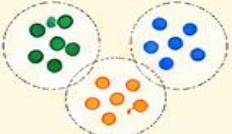
In simple words: It detects data points that do not fit the normal pattern.

Common Techniques: Isolation Forest, One-Class SVM, LOF

ML TASKS AND THE TYPES OF ML ALGORITHMS



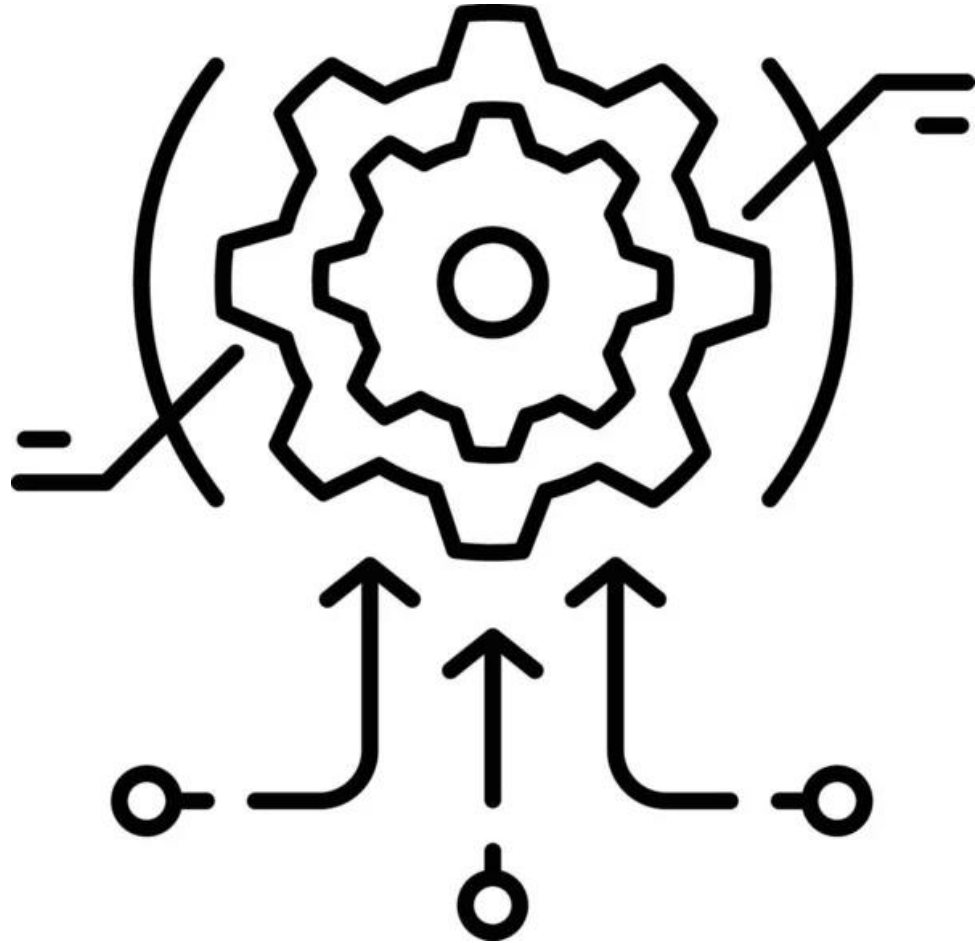
Different ML **tasks** are solved using different **types of algorithms** depending on the **data** and **goal**.

ML TASK	GOAL	TYPE OF ML	DATA / LABEL INFORMATION	EXAMPLE PROBLEM	COMMON ALGORITHMS
 1. CLASSIFICATION	Predict a category or class label.	Supervised Learning 	• Uses labeled data • Target variable is categorical	• Email spam or not spam • Disease: Yes / No • Image: Cat or Dog	• Logistic Regression • Decision Tree • Random Forest • Support Vector Machine (SVM) • k-Nearest Neighbors (k-NN)
 2. REGRESSION	Predict a continuous numeric value.	Supervised Learning 	• Uses labeled data • Target variable is numeric (continuous)	• House price prediction • Temperature prediction • Sales forecasting	• Linear Regression • Polynomial Regression • Ridge / Lasso Regression • Support Vector Regression (SVR) • Decision Tree Regressor
 3. CLUSTERING	Group similar data points together.	Unsupervised Learning 	• Uses unlabeled data • No target variable	• Customer segmentation • Grouping documents • Market basket analysis	• K-Means • Hierarchical Clustering • DBSCAN • Mean Shift • Gaussian Mixture Models (GMM)
 4. DIMENSIONALITY REDUCTION	Reduce the number of features while keeping important information.	Unsupervised Learning 	• Uses unlabeled data • No target variable	• Visualizing high-dimensional data • Reduce computation time • Remove redundant features	• PCA (Principal Component Analysis) • t-SNE • LDA (Linear Discriminant Analysis) • Autoencoders
 5. ANOMALY DETECTION	Identify rare, unusual or suspicious data points.	Unsupervised Learning 	• Uses unlabeled data • No target variable (only normal data usually available)	• Fraud detection • Network intrusion detection • Fault detection in machines	• Isolation Forest • One-Class SVM • LOF (Local Outlier Factor) • Autoencoders

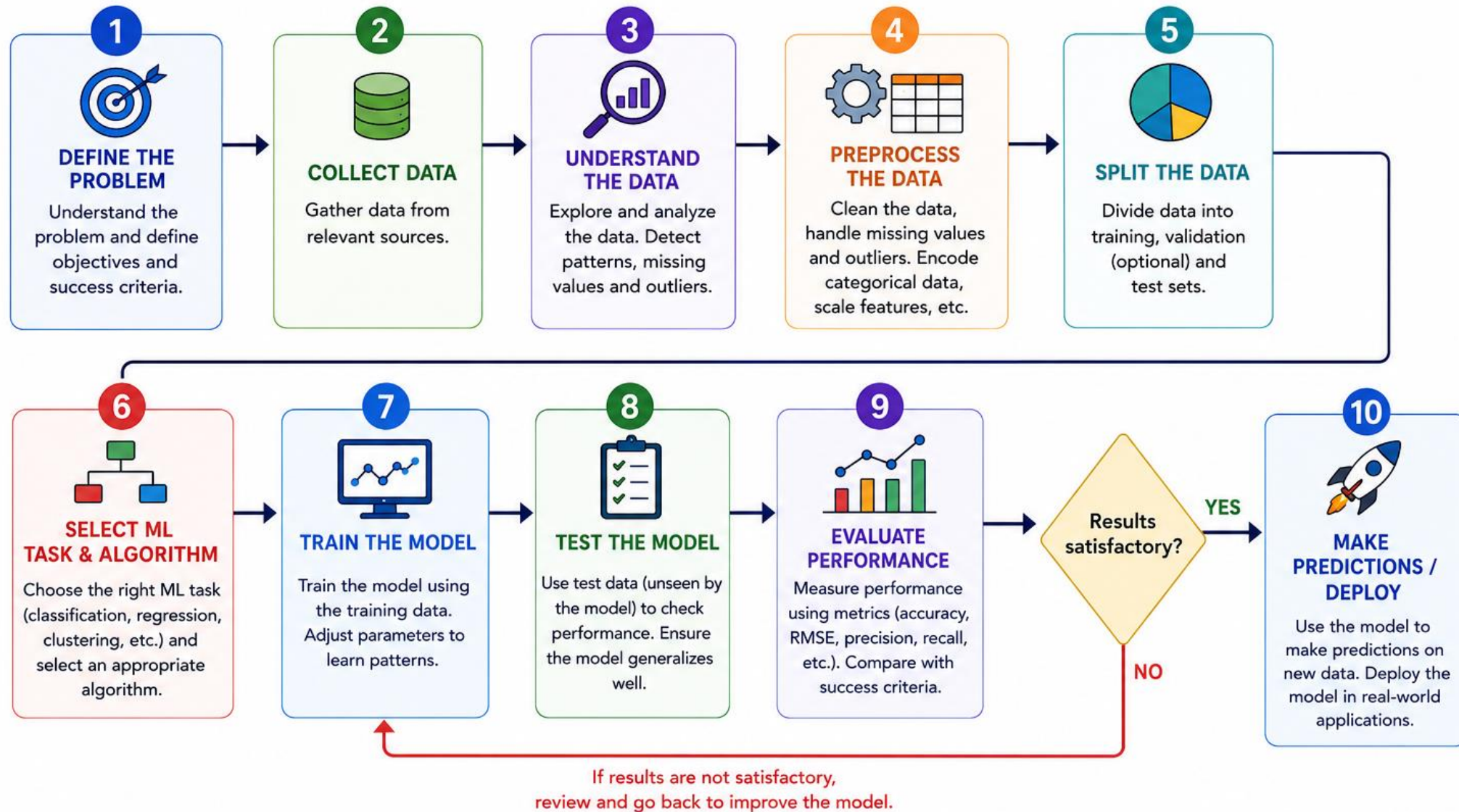


SUMMARY: 1 & 2 use **labeled data** (Supervised Learning). 3, 4 & 5 use **unlabeled data** (Unsupervised Learning).

Machine Learning Process Flow

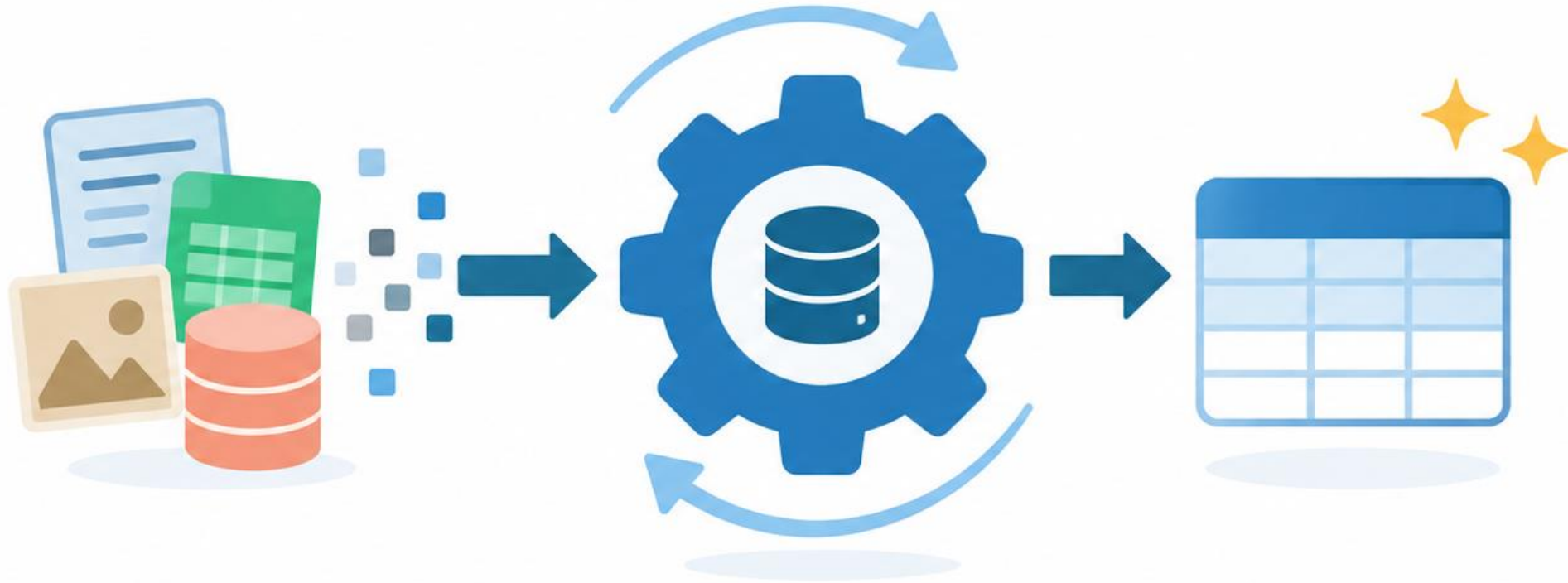


MACHINE LEARNING PROCESS FLOW



NOTE: This process is **iterative**. Based on results, you may go back to earlier steps (especially 4, 5, 6 or 7) to improve data, try different algorithms or tune parameters.

Data Preprocessing



Data Understanding: Relevance of Data

Check if the available data is useful and sufficient for the task.

1

What data is available for the task?



- Identify the data sources.
- What information (attributes) is currently available?

Example: Student records with age, gender, attendance, marks.

2

Is this data relevant?



- Check if the available data helps to answer the problem.
- Remove irrelevant data.

Example: For predicting student performance, age and marks are relevant, while blood group may not be relevant.

3

Is additional relevant data available?



- Can we collect more useful data from other sources?
- Will additional data improve the analysis?

Example: In addition to student records, we may include previous academic performance or library usage.

4

How much historical data is available?



- Check the time period covered by the data.
- More historical data helps in finding trends and making better predictions.

Example: Attendance data for the last 3 years.



Use relevant and sufficient data → Better analysis and decisions

Data Understanding: Quantity of Data

NUMBER OF INSTANCES (Records / Objects)

- Number of rows in the dataset.
- Each row represents one object or record.

Example:

1,000 student records.

Too few records → results may be less reliable.

NUMBER OF ATTRIBUTES (Fields / Features)

- Number of columns (features) in the dataset.
- Each attribute describes a property of the object.

Example:

10 attributes (Age, Gender, Attendance, Marks, etc.).

Too many features → consider feature selection / reduction.



Very unbalanced data → consider suitable sampling.

Why Data Pre-processing?

Real-world data is ...



Incomplete
(missing values)



Noisy
(errors, outliers)



Inconsistent
(different formats)



Redundant
(duplicates)



Often not in a suitable format for ML models
(categorical, text, etc.)

Pre-processing

cleans,
transforms
and prepares
the data

If we do not preprocess, it can lead to ...



In healthcare, noisy or missing data can lead to incorrect disease predictions.

Risk to patient safety



In finance, inconsistent or duplicate data can result in false fraud alerts or missed fraud cases.

Financial losses



In autonomous vehicles, poorly processed sensor data can cause wrong object detection.

Safety hazards

Good data preparation → More reliable, fair and accurate AI systems.

Why Data Pre-processing?

A real-world dataset may be incomplete before analysis.

Item	Price	Units Sold	Session
TV	7200	--	All
Fan	480	27	Summer
Tube light	54	30	All
AC	27000	38	All
Fridge		40	Summer
Switches	58	35	Summer
2 mm Wire	520		All
Backup Fan		48	Winter
Regulator	83	50	All
Bulb	87	37	Rainy Season

Missing / unavailable values → preprocessing is needed before ML analysis

What do you mean by Data Pre-processing?

Data pre-processing is the process of **cleaning, transforming** and **preparing raw data** to make it **suitable for analysis** and **machine learning**.



Cleaning and exploring data for analysis

Remove errors, handle missing values and understand the data.



Prepping data for modeling

Prepare data in a form that can be used by algorithms.



ML models generally require suitable numerical input

Convert categorical data to numerical form if required.



Transforming raw data into an understandable / usable format

Change data into a consistent and meaningful representation.



Resolving data_quality issues

Identify and fix problems such as outliers, inconsistent data and duplicates.

RAW DATA

(Noisy, incomplete, inconsistent)



DATA PRE-PROCESSING

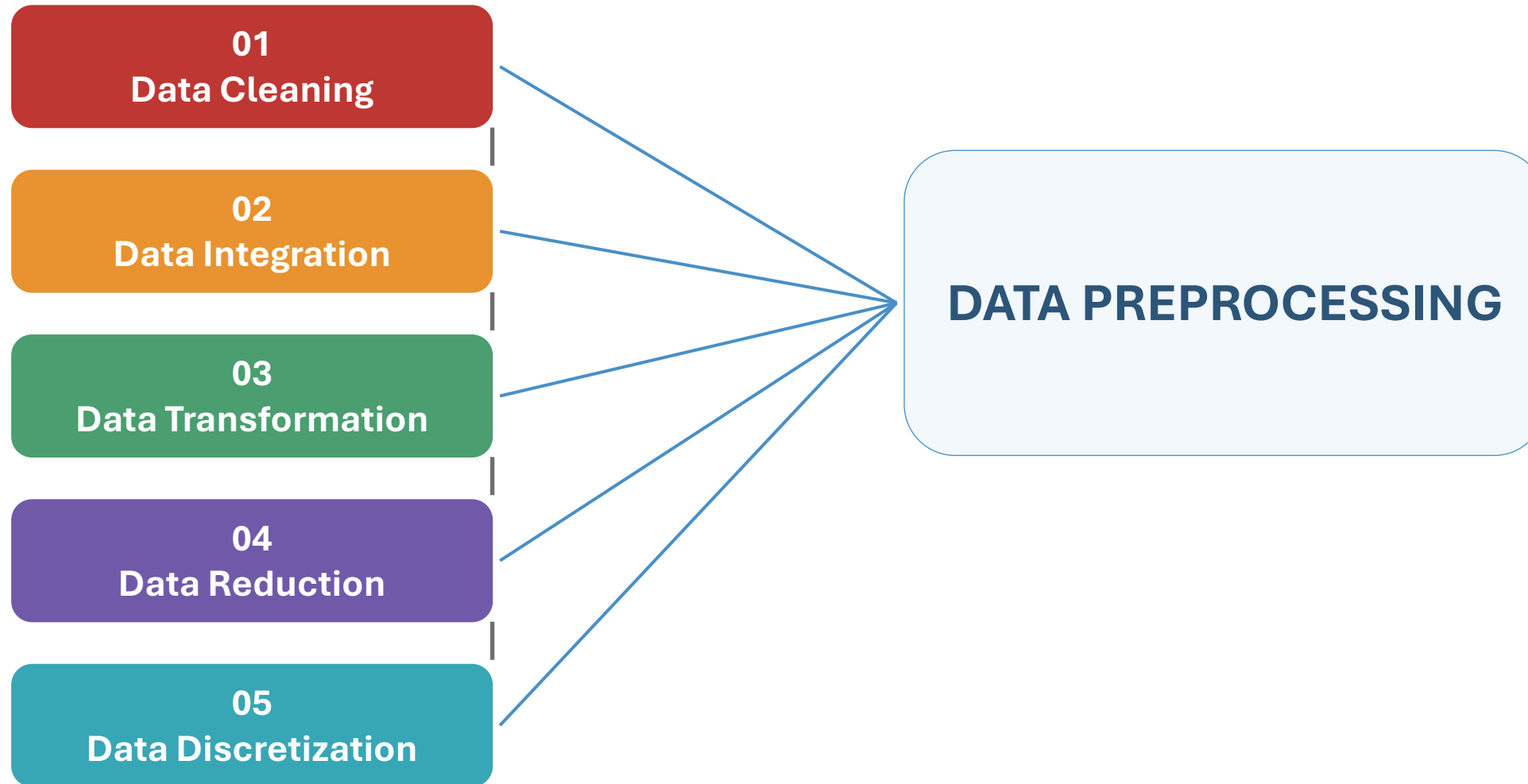
(Clean, transform, prepare)



UNDERSTANDABLE / USABLE DATA

(Ready for analysis and modeling)

Data Pre-processing Steps



Data Cleaning

Identify and fix data quality problems to make the data reliable and usable.

Data Cleaning Tasks

1 Fill in missing values

Fill in values that are missing or blank.

Example

ID	Age	Salary
101	25	30,000
102	?	32,000
103	28	35,000



Action

Fill with an appropriate value (e.g., mean) or remove the record.

2 Identify outliers and handle noisy data

Find unusual values and correct or remove incorrect data.

Example

ID	Age	Salary	
103	250	35,000	(outlier)
105	35	3,200	(noise)



Action

Investigate and correct, transform, or remove.

3 Correct inconsistent data

Make different forms of the same data uniform.

Example

ID	Gender
101	Male
103	M
104	male



Action

Standardize to a consistent format (e.g., "Male").

4 Resolve redundancy caused by data integration

Remove repeated or unnecessary data.

Example

ID	Age	Salary	
104	28	30,000	
104	28	30,000	(duplicate)



Action

Identify and remove duplicate records.

Clean data → Better analysis → Reliable decisions

Identify the Data Problem					
ID	Age	Gender	Designation	Salary	Data Problem
101	25	Male	Assistant Professor	₹30,000	
102	?	Female	Assistant Professor	₹32,000	
103	250	M	Assistant Professor	₹35,000	
104	28	male	Assistant Professor	₹30,000	
105	35	Female	Assistant Professor	₹250	
104	28	male	Assistant Professor	₹30,000	

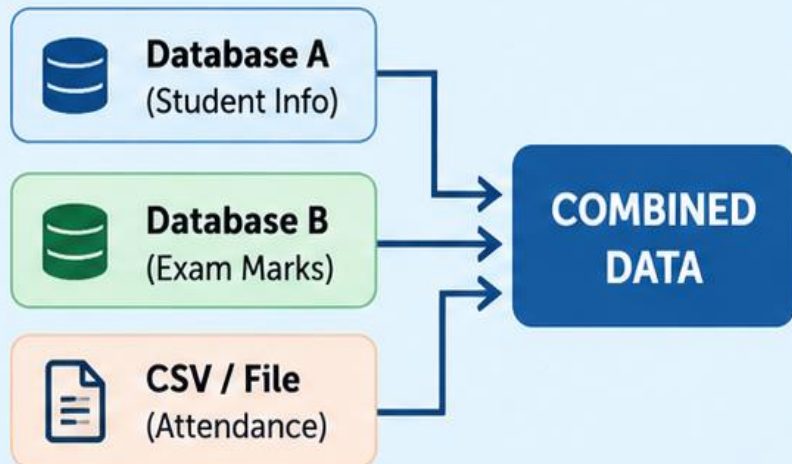
- | Data Problem |
|----------------------------|
| 1. Noise / Error |
| 2. Duplicate Record |
| 3. Missing Value |
| 4. Outlier / Invalid Value |
| 5. Normal |
| 6. Inconsistent Value |



Data Integration, Transformation & Reduction

1. DATA INTEGRATION

Combine data from multiple sources

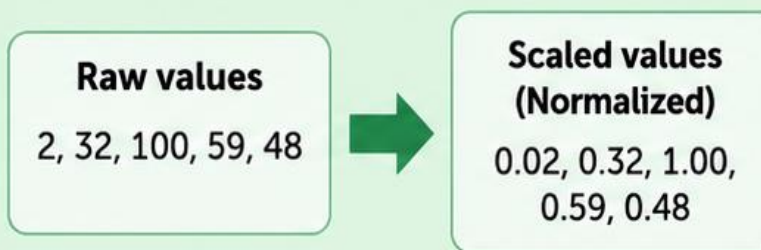


- Integrate data from different sources (e.g., databases, files, APIs).
- Resolve conflicts and remove redundancy.
- Create a single, consistent dataset for analysis.

Example: Combine student details, exam marks and attendance into one dataset using Student ID.

2. DATA TRANSFORMATION

Convert data into a suitable form



- Change the representation or scale of data.
- Create new features if needed.
- Common methods:
 - Normalization / standardization
 - Feature construction
 - Aggregation

Example: Scale numerical values (e.g., age, salary) to a common range like 0 – 1.

3. DATA REDUCTION

Keep a smaller representation



- Reduce the volume or complexity of data while retaining important information.
- Remove irrelevant or redundant features.
- Common techniques:
 - Feature selection
 - Dimensionality reduction (e.g., PCA)
 - Sampling / aggregation

Example: Select the most important features and reduce 126 attributes to 115.

Better data → Better analysis → Better decisions

Data Discretization

Transforms continuous (quantitative) data into qualitative intervals (categories).

1. Original Data (Continuous Values)

Humidity (%)
85
80
75
72
68
65
60
55

2. Discretization (Define Intervals)

LOW = 60 – 69

NORMAL = 70 – 79

HIGH = 80+

3. Discretized Data (Categories)

Humidity (%)	Category
85	High
80	High
75	Normal
72	Normal
68	Low
65	Low
60	Low
55	Low

What is Discretization?

- Discretization converts continuous numerical values into a small number of intervals or categories.
- It helps in simplifying data and is useful for some data mining and machine learning techniques.
- Example: Converting exact humidity values into Low, Normal and High.

Why use Discretization?

- Reduces the effect of small variations or noise.
- Makes the data easier to understand and analyze.
- Useful for algorithms that work with categorical data.
- Helps in creating simple, interpretable models.

Continuous values → Intervals / Categories

Discretization = converting continuous numerical values into groups/categories (bins).

Feature Selection

Select the most relevant features from the available features.

Data Reduction

Example Dataset: Student Performance

ID	Age	Attendance (%)	Internal Marks	Hobby	Final Marks
1	20	85	78	Music	82
2	21	92	88	Reading	90
3	20	75	65	Sports	68
4	22	90	92	Dance	94
5	21	60	55	Travel	58

Available Features (all attributes)

Age

Attendance

Internal
Marks

Hobby



Selected Features (relevant for prediction)

Attendance

Internal Marks

Removed Features (less relevant)

Age

Hobby



Feature selection does not create new features.

It simply chooses the most relevant features from the existing features.

Feature Extraction

Create new features from the existing features.

Data Reduction

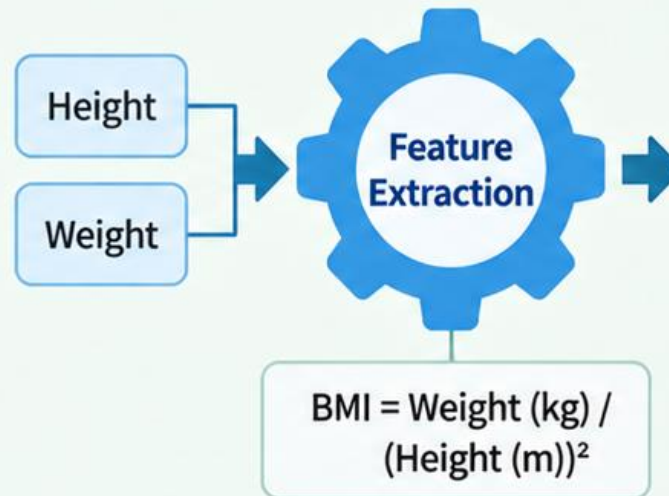
Original Features

Example: Health Data

ID	Height (cm)	Weight (kg)	Age
1	160	50	25
2	170	65	22
3	155	45	20
4	180	80	28
5	165	60	24

Feature Extraction

Combine or transform existing features to create new feature(s).



Dataset with New Feature

ID	Height (cm)	Weight (kg)	BMI (kg/m ²)	Age
1	160	50	19.5	25
2	170	65	22.5	22
3	155	45	18.7	20
4	180	80	24.7	28
5	165	60	22.0	24



Feature extraction creates new feature(s) from the existing features.

It transforms or combines the original features to represent the important information more compactly.

Pre-processing Techniques in ML

Methods Used

01

Data Cleaning

Handle missing/incorrect/noisy data.

Missing Numerical Values

Fill missing values using statistical measures.

Method	What it does	When to use
Mean Imputation	Fills missing value with the average of the available values.	When data is approximately normal and without extreme outliers.
Median Imputation	Fills missing value with the middle value of the available data.	When data contains outliers or is not normally distributed.

Example

Age (years)				
20	22	?	24	25



Missing value can be filled using **Mean** or **Median**.

03

Data Transformation

Convert data to a suitable form for ML models.

Scaling Numerical Features

Bring features to a common scale.

Method	What it does	When to use
Normalization (Min-Max Scaling)	Rescales values to a fixed range, usually 0 to 1 .	When you want all features in the same range.
Standardization (Z-score Scaling)	Transforms values based on mean and standard deviation (mean = 0, std = 1).	When features have different scales and you want them to have zero mean and unit variance.

Example

Feature	Original Values	After Normalization (0 – 1)	After Standardization (mean = 0, std = 1)
Age (years)	20 – 60	0.00 – 1.00	≈ -1.2 – 1.3
Salary (₹)	20,000 – 2,00,000	0.00 – 1.00	≈ -1.1 – 1.4



Key Takeaway

Use **Mean/Median** at the Data Cleaning step for missing values.

Use Normalization/Standardization at the Data Transformation step to bring features to a common scale.

02

Data Integration

Combine data from multiple sources.

Integrate data from different sources

Merge data that refer to the same entity from multiple datasets using a common key.



Database A
Student Info

Student ID	Name
S001	Alice
S002	Bob
S003	Carol



Database B
Exam Marks

Student ID	Marks
S001	85
S002	78
S003	92



CSV / File
Attendance

Student ID	Attendance
S001	95%
S002	88%
S003	92%



COMBINED DATA

Common Methods

Method	What it does
Merge / Join	Combine datasets using a common key (e.g., Student ID).
Concatenation	Append datasets (same structure, e.g., multiple files).
Aggregation	Combine data by summarizing (e.g., average, sum, count).

Example: Student Data Integration

Combine student information, exam marks and attendance using Student ID.

Before Integration (Separate Datasets)

Student Info	
Student ID	Name
S001	Alice
S002	Bob
S003	Carol

Exam Marks	
Student ID	Marks
S001	85
S002	78
S003	92

Attendance	
Student ID	Attendance
S001	95%
S002	88%
S003	92%



After Integration (Combined Dataset)

Student ID	Name	Marks	Attendance
S001	Alice	85	95%
S002	Bob	78	88%
S003	Carol	92	92%

Example

Combine student details, exam marks and attendance using **Student ID**.



Key Takeaway

Data Integration → Combine data from multiple sources into one consistent dataset.

04

Data Reduction

Reduce the data size by removing irrelevant or redundant information.

Common Techniques

Keep only useful features or a representative subset of data.

Technique	Methods / Examples
Feature Selection	• Filter Method (e.g., Correlation, Chi-square, Mutual Information) • Wrapper Method (e.g., Recursive Feature Elimination – RFE) • Embedded Method (e.g., Lasso, Decision Tree, Random Forest)
Feature Extraction	• Principal Component Analysis (PCA) • Linear Discriminant Analysis (LDA) • Independent Component Analysis (ICA) • Autoencoders (Deep Learning)
Sampling	• Random Sampling • Stratified Sampling • Cluster Sampling • Under-sampling / Over-sampling (e.g., for imbalanced data)

05

Data Discretization

Convert continuous numerical data into a small number of discrete values (intervals or categories).

Common Techniques

Convert continuous values into meaningful categories.

Technique	Methods / Examples
Binning (Equal-width)	• Divide the range into equal-sized intervals (bins). • Example: 0–10, 10–20, 20–30, ...
Binning (Equal-frequency)	• Each bin has approximately the same number of data points. • Useful when data is not uniformly distributed.
Binning (Entropy/ChiMerge)	• Uses entropy or chi-square to create optimal bins. • Common in decision tree algorithms (e.g., ChiMerge).
Label Encoding (for discretization)	• Assign integer labels to discrete categories (if required for ML models). • Example: Low = 0, Medium = 1, High = 2.



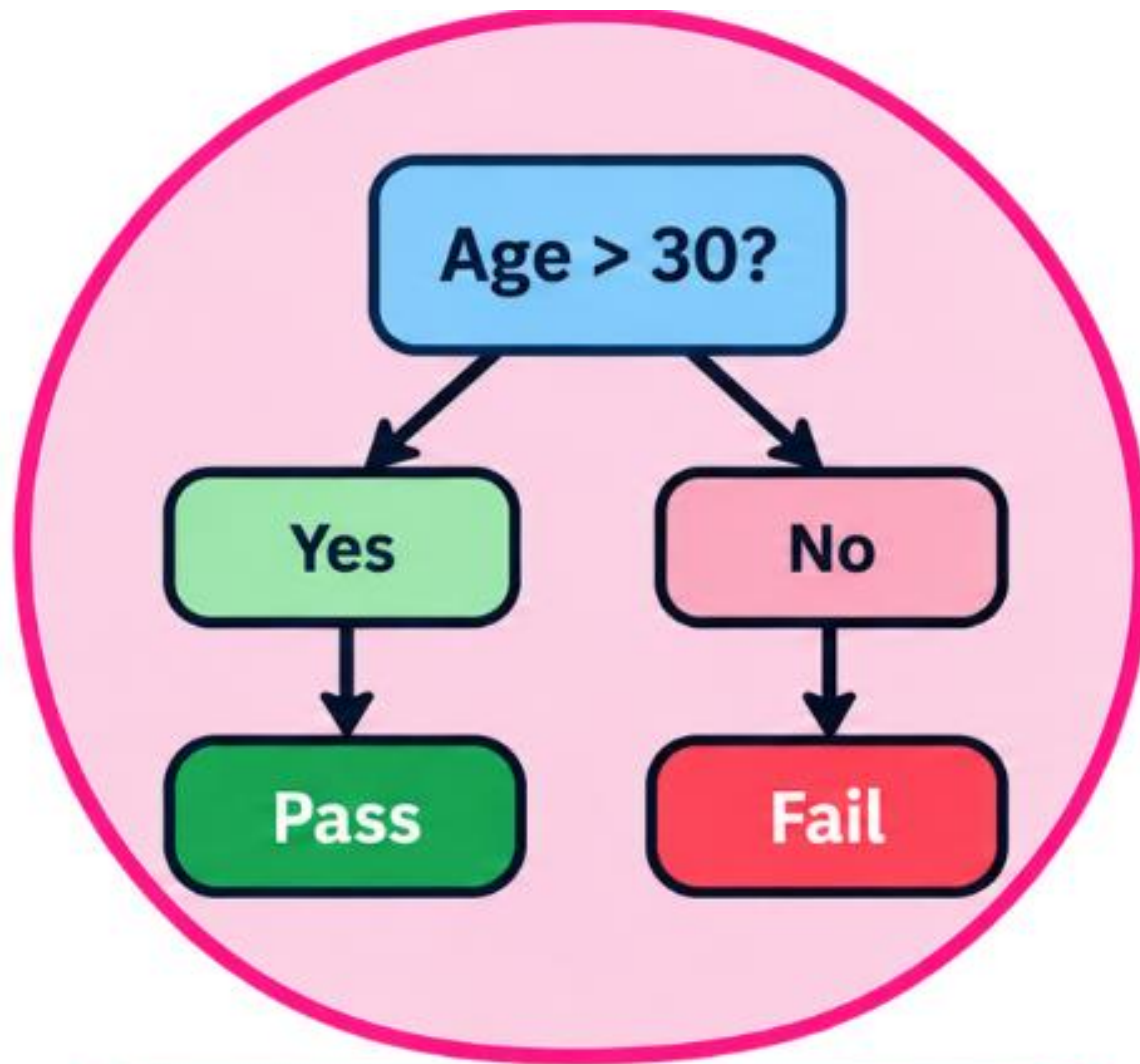
Key Takeaway

Data Reduction → Feature Selection (Filter, Wrapper, Embedded), Feature Extraction (PCA, LDA, ...), Sampling.

Data Discretization → Equal-width Binning, Equal-frequency Binning, Entropy/ChiMerge, Label Encoding.



ML Algorithms



Decision Tree (DT)



Decision Tree – An Everyday Example

Slide 1/2

We all make decisions in daily life!

If you want to go outside today, what will you check?

Should I
go outside
today?



- Is it raining?
- Is it sunny?
- Is it too hot?
- Do I have an umbrella?
- ...

A Simple Decision Tree



You are making a decision by asking questions one after another.
This is a **Decision Tree**!



Decision Tree – Key Concepts and Connection to ML

Slide 2/2

1 Why is it called a Tree?

The diagram starts from one point, then splits into different paths, just like the branches of a tree.

2 Now connect it to Machine Learning

Until now, we created the questions ourselves. But in Machine Learning, we want the computer to **learn these questions from data**.

Weather	Play Cricket
Rainy	No
Rainy	No
Sunny	Yes
Sunny	Yes
Cloudy	Yes

Can you find a simple rule from this data?

Rainy → No
Sunny → Yes



3 Simple Definition

Decision Tree

A supervised machine learning algorithm that makes predictions by following a sequence of decisions learned from data.



Key Idea

The computer automatically finds the best questions to make decisions from the training data.

What are we trying to **measure**?

The goal is to find the best split!



Remember our goal:

The Decision Tree wants to find a split that creates groups containing **mostly one class**.



A Very Good Split

Split based on some feature

Group 1

Yes Yes Yes

Group 2

No No No

- Each group contains mostly one class.
- These groups are **pure**.



A Poor Split

Split based on some feature

Group 1

Yes No Yes

Group 2

No Yes No

- Each group contains a mix of classes.
- These groups are **mixed**.

So, the mathematical measures help the tree answer:

“How good is this split?”

There are three important concepts:

1

Entropy

Measures uncertainty or impurity in a group.

2

Gini Impurity

Another measure of impurity.

3

Information Gain

Measures how much a split reduces impurity.

In the next slides, we will look at each of these in detail.



Entropy – Measuring Uncertainty in Data

Lower entropy
means purer data!



Entropy

Entropy is a measure of **uncertainty or impurity** in a dataset. It tells us how mixed the classes are.

Example 1 – Pure Group

Yes

Yes

Yes

Yes

Are you confused about the class?

No!

We know it is completely Yes.

Uncertainty = 0

Entropy = 0

Example 2 – Mixed Group

Yes

No

Yes

No

Are you confused about the class?

Yes!

We don't know whether the next item will be Yes or No.

**More mixed →
More uncertainty →
Higher entropy**

Summary

Group	Data	Entropy (Uncertainty)
Pure	Yes, Yes, Yes	0
Pure	No, No, No	0
Mixed	Yes, No, Yes, No	High (maximum is 1 for binary class)

Entropy Formula

$$\text{Entropy}(S) = - \sum_i p_i \log_2(p_i)$$

Where:

- p_i = probability (proportion) of each class
- $\log_2$ = logarithm with base 2
- $\sum$ = add the values for all classes

For two classes (Yes and No):

$$\text{Entropy} = -(p_{\text{Yes}} \log_2 p_{\text{Yes}} + p_{\text{No}} \log_2 p_{\text{No}})$$

Example Calculation

Suppose we have:

Yes = 4

No = 0

Total = 4

$$\left. \begin{aligned} p_{\text{Yes}} &= \frac{4}{4} = 1 \\ p_{\text{No}} &= \frac{0}{4} = 0 \end{aligned} \right\} \rightarrow$$

**Entropy
= 0**

Because there is **no uncertainty**. The group is pure (all belong to the same class).



Key Takeaway

Entropy measures how **uncertain or mixed** the data is. **Lower entropy** means purer data.

Entropy vs Gini Impurity – Same Goal, Different Way

Both measure
impurity!



Same Goal

Both Entropy and Gini Impurity are used to measure how **impure or mixed** a group is and to find the **best split** in a Decision Tree.

Entropy

- Measures **uncertainty** in the data.
- Uses **logarithm**.
- More sensitive to small changes in class distribution.

$$\text{Entropy} = - \sum_i p_i \log_2(p_i)$$

Gini Impurity

- Measures **impurity** or misclassification probability.
- Uses **squared probabilities** (no logarithm).
- Usually faster to compute.

$$\text{Gini} = 1 - \sum_i p_i^2$$

Key Differences (Short & Simple)

Aspect	Entropy	Gini Impurity
Measures	Uncertainty	Impurity
Formula	$-\sum p_i \log_2(p_i)$	$1 - \sum p_i^2$
Uses logarithm?	Yes	No
Sensitivity	More sensitive to small changes	Less sensitive
Computation	Slightly slower	Faster



- Use **Entropy** when you want a **more detailed measure** of uncertainty (often used in ID3 algorithm).
- Use **Gini Impurity** when you want a **simpler and faster** measure (commonly used in CART algorithm, e.g., scikit-learn).

Conclusion



Both work well!

They are just different ways to measure the **same thing** — **impurity** in the data.

Same destination ... different paths!



Information Gain

How much uncertainty did we reduce?

Choose the split
that reduces uncertainty
the most!



What is Information Gain?

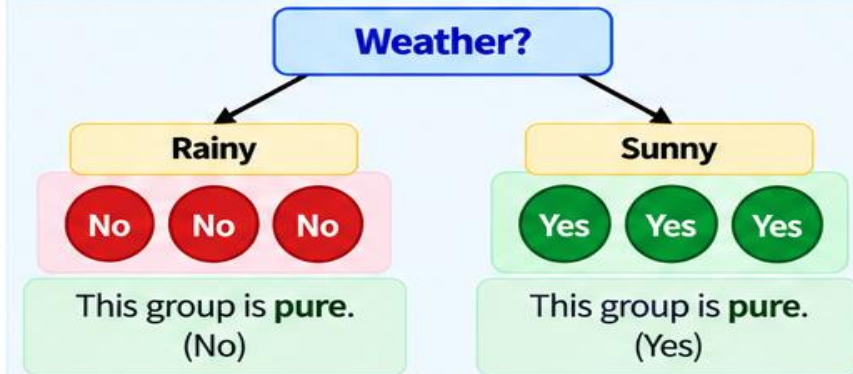
Information Gain tells us how much **uncertainty is reduced** after we split the data. It helps the tree to choose the **best question (feature)** for splitting.

1. Before Split (Mixed Data)



The data is mixed.
There is **high uncertainty**

2. Split by Weather



3. After Split (Lower Uncertainty)

The groups are now pure.
Uncertainty has **reduced significantly**.



So, this split gives
high Information Gain
(a good split).

Information Gain Formula

$$\text{Information Gain} = \text{Entropy (before)} - \text{Entropy (after)}$$

It is the **reduction in uncertainty** after the split.

Think of it like this:

Before split
(High uncertainty)

Split using a feature
(e.g., Weather)

After split
(Lower uncertainty)

The amount of
reduction in
uncertainty is the
Information Gain.



Key Takeaway: Choose the split that gives the **highest Information Gain**, because it creates **purier groups**.

Decision Tree Example (1/2)

Step-by-step mathematical derivation using Entropy and Information Gain



1. Dataset

No.	Weather	Wind	Play Cricket
1	Sunny	Weak	Yes
2	Sunny	Strong	No
3	Sunny	Weak	Yes
4	Rainy	Weak	No
5	Rainy	Strong	No
6	Rainy	Weak	No
7	Cloudy	Weak	Yes
8	Cloudy	Strong	Yes

Total: 8 samples

Yes = 4 No = 4

2. Entropy of the root node

$$\text{Entropy}(S) = - \sum p_i \log_2(p_i)$$

$$p_{\text{Yes}} = \frac{4}{8} = 0.5 \quad p_{\text{No}} = \frac{4}{8} = 0.5$$

$$\begin{aligned} \text{Entropy}(S) &= - (0.5 \log_2 0.5 + 0.5 \log_2 0.5) \\ &= - (0.5(-1) + 0.5(-1)) = 1 \end{aligned}$$

Entropy(S) = 1 — (completely mixed)

3. Try splitting by Weather

Calculate entropy for each group.

Sunny (3 samples)

Yes = 2 No = 1

$$p_{\text{Yes}} = \frac{2}{3} \quad p_{\text{No}} = \frac{1}{3}$$

Entropy(Sunny)

$$= - \left(\frac{2}{3} \log_2 \frac{2}{3} + \frac{1}{3} \log_2 \frac{1}{3} \right)$$

= **0.918** (approx)

Rainy (3 samples)

Yes = 0 No = 3

$$p_{\text{Yes}} = 0 \quad p_{\text{No}} = 1$$

Entropy(Rainy) = **0**

Cloudy (2 samples)

Yes = 2 No = 0

$$p_{\text{Yes}} = 1 \quad p_{\text{No}} = 0$$

Entropy(Cloudy) = **0**

4. Entropy after split (weighted average)

$$\text{Entropy(after)} = \frac{3}{8} (0.918) + \frac{3}{8} (0) + \frac{2}{8} (0) = \mathbf{0.344}$$

5. Information Gain for Weather

$$\text{IG(Weather)} = \text{Entropy(before)} - \text{Entropy(after)} = 1 - 0.344 = \mathbf{0.656}$$

IG(Weather) $\approx$ 0.656 $\rightarrow$ Good split!

Decision Tree Example (2/2)

Compare features, choose the best split and build the tree

Higher
Information Gain
= Better split!

6. Try splitting by Wind

Calculate entropy for each group.

Weak (5 samples)

Yes = 3 No = 2

$$P_{\text{Yes}} = \frac{3}{5} \quad P_{\text{No}} = \frac{2}{5}$$

$$\text{Entropy(Weak)} = 0.971 \text{ (approx)}$$

Strong (3 samples)

Yes = 0 No = 3

$$P_{\text{Yes}} = 0 \quad P_{\text{No}} = 1$$

$$\text{Entropy(Strong)} = 0$$

$$\text{Entropy(after)} = \frac{5}{8} (0.971) + \frac{3}{8} (0) = 0.607$$

$$\text{IG(Wind)} = 1 - 0.607 = 0.393$$

7. Compare Information Gain

Feature	Information Gain
Weather	0.656
Wind	0.393

Choose Weather as the root split
(since it has higher Information Gain)

8. Build the Decision Tree

After splitting by Weather, the Sunny group is still mixed, so we split it further using Wind.



Final Decision Tree

- **Weather** is the best first question (because it reduces uncertainty the most).
- If Weather = Rainy → Play = No
- If Weather = Cloudy → Play = Yes
- If Weather = Sunny, then check Wind:
 - Wind = Weak → Yes
 - Wind = Strong → No



Key Takeaways

- 1 **Entropy / Gini** → measure how impure a node is.
- 2 **Information Gain** → measures how much impurity is reduced after a split.
- 3 Choose the split with the **highest Information Gain**.
- 4 Repeat the process for remaining impure nodes.



The tree keeps asking the best questions and continues splitting until the groups become pure.

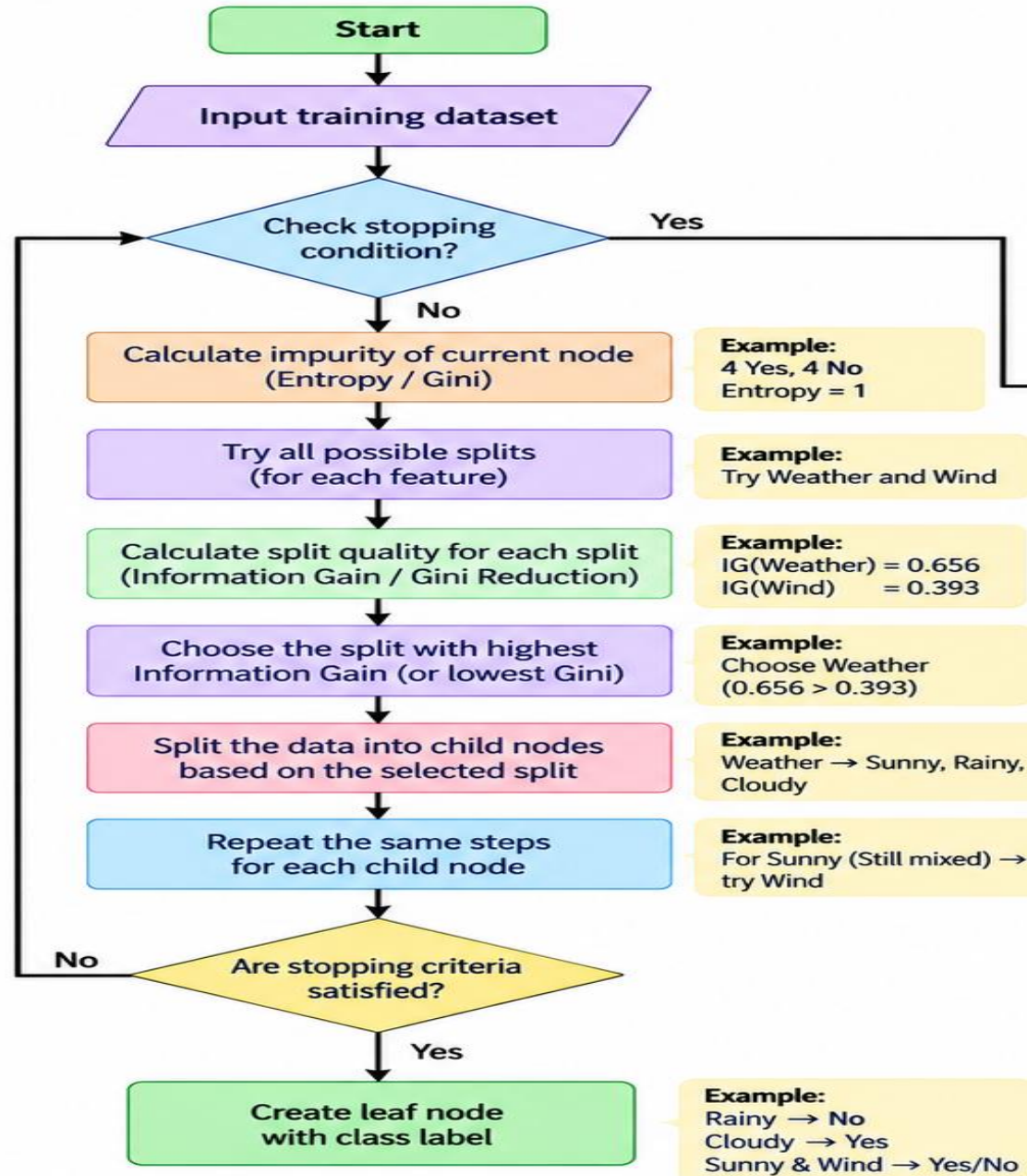
Simple idea:

Try a split → Measure impurity
→ Choose the best → Repeat.

Decision Tree Algorithm

(Flowchart with Example)

- Step 1:** Input training dataset
- Step 2:** Check stopping condition
- Step 3:** Calculate node impurity (Entropy / Gini)
- Step 4:** Evaluate possible splits
- Step 5:** Calculate split quality (Information Gain / Gini Reduction)
- Step 6:** Select the best split
- Step 7:** Split the dataset
- Step 8:** Recursively repeat for child nodes
- Step 9:** Stop when criteria are satisfied
- Step 10:** Create leaf nodes and assign predictions
- Step 11:** Return the Decision Tree



Training Dataset: Play Cricket?			
No.	Weather	Wind	Play Cricket
1	Sunny	Weak	Yes
2	Sunny	Strong	No
3	Sunny	Weak	Yes
4	Rainy	Weak	No
5	Rainy	Strong	No
6	Rainy	Weak	No
7	Cloudy	Weak	Yes
8	Cloudy	Strong	Yes

Create leaf node with class label (Majority class)

Example:
If all samples are Yes (or all No), stop and create a leaf node.

Return Decision Tree

Step 11: Return the Decision Tree

Final Decision Tree for the Example



Step	What is happening?	Simple Example
1. Input Training Dataset	Start with training data containing features and known target labels.	Weather, Wind → Play?
2. Check Stopping Condition	Check whether we should stop splitting this node.	If all are Yes → stop.
3. Calculate Node Impurity	Measure how mixed the classes are using Entropy or Gini.	4 Yes + 4 No → Entropy = 1
4. Evaluate Possible Splits	Try different features/questions that could split the data.	Try Weather and Wind.
5. Calculate Split Quality	Measure how much each split reduces impurity using Information Gain or Gini Reduction.	Weather → IG = 0.656
6. Select the Best Split	Choose the split that gives the best impurity reduction.	Weather > Wind → choose Weather.
7. Split the Dataset	Divide the data into child groups according to the selected split.	Sunny / Rainy / Cloudy
8. Repeat for Child Nodes	Apply the same process to each child group.	Sunny is mixed → try Wind.
9. Stop When Criteria Are Satisfied	Stop splitting when the stopping condition is reached.	Rainy → all No → stop.
10. Create Leaf Nodes	Assign the final class/prediction to each leaf.	Rainy → No; Cloudy → Yes.
11. Return the Decision Tree	The completed tree is ready to make predictions.	Weather → Wind → Yes/No

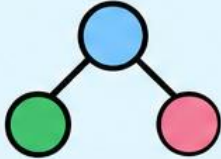


Variants of Decision Trees

Different algorithms, same idea – split the data to make decisions!



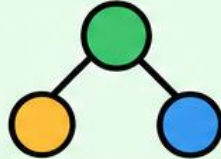
ID3



- Uses Information Gain (Entropy) to choose splits.
- Works with categorical data only.
- Builds the tree in a top-down way (greedy approach).

Good for simple classification problems with categorical data.

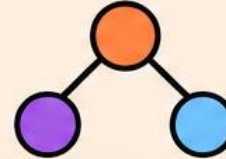
C4.5



- Improved version of ID3.
- Uses **Gain Ratio** to choose splits.
- Handles both categorical and numerical data.
- Can handle missing values.

More flexible than ID3 and popular in practice.

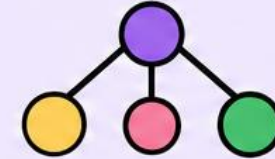
CART



- Uses **Gini Index** (for classification) or Mean Squared Error (for regression).
- Can be used for both classification and regression.
- Generally creates binary splits (two branches).

Widely used and forms the basis of many modern models (e.g., Random Forest).

CHAID



- Uses chi-square test to choose splits.
- Works mainly with categorical data.
- Can create multi-way splits (more than two branches).

Often used in market research and social sciences.



Key Takeaway

“ All these variants follow the same basic idea, but they differ in how they choose splits and the type of data they handle. ”



Decision Trees – Pros and Cons

Simple • Interpretable • Powerful (with some limitations)



Advantages



Easy to understand

→ Looks like a flowchart of simple questions.



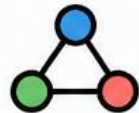
Works with different types of data

→ Can handle numerical and categorical features.



Little preprocessing needed

→ No need for normalization or standardization.



Can model complex patterns

→ Can make non-linear decisions.

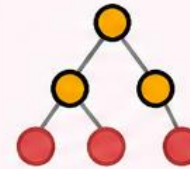


Fast predictions

→ Once the tree is built, predictions are quick.



Disadvantages



Can overfit

→ A very deep tree may memorize the training data.



Can be unstable

→ Small changes in data may produce a different tree.



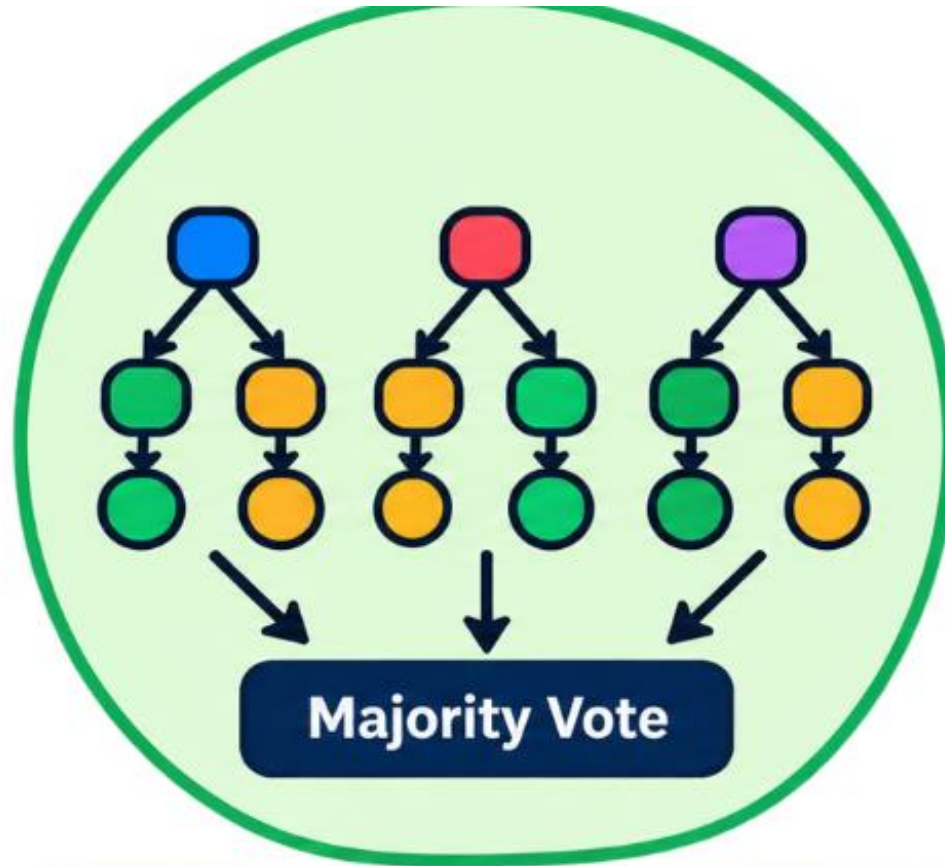
May not always give the best performance

→ Other methods can sometimes perform better.



Key Takeaway

“ Decision Trees are simple and easy to interpret, but they can overfit. ”



Random Forest (RF)

Why Do We Need Random Forest?

A single decision tree can work, but it has some important limitations!



What is the limitation of a Single Decision Tree?

- 1 A Decision Tree learns **one set of decision rules** from the training data.
- 2 Small changes in the training data can produce a **very different tree**.
- 3 A deep tree can memorize the training data → **Overfitting**.
- 4 A single tree may be strongly influenced by **noisy data or unusual observations**.
- 5 Therefore, its predictions may have **high variance**.

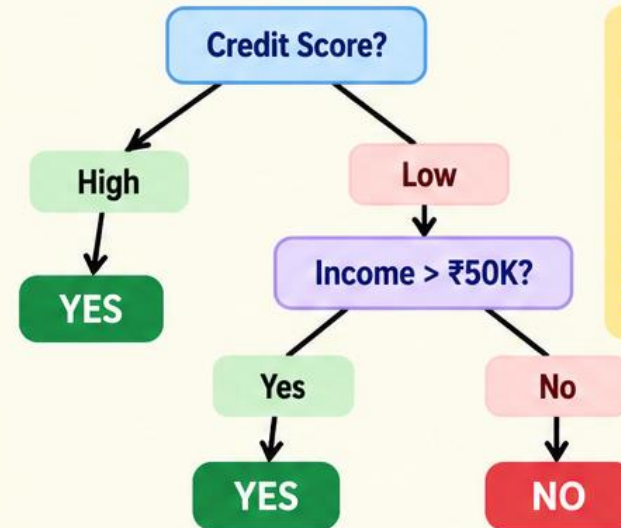


Example: Loan Approval

Suppose we want to predict:

“Should a bank approve a loan?”

A single Decision Tree might learn:



But if a few training records change, the tree might learn **completely different rules**.



Can we build a better model?



This is the idea behind **Random Forest!**





How Does Random Forest Help?

Many trees working together give a stronger and more reliable prediction!

Stronger
Together!



The Basic Idea

1

Instead of depending on **ONE** tree,

2

Random Forest builds **MANY Decision Trees** (using different samples and features).

3

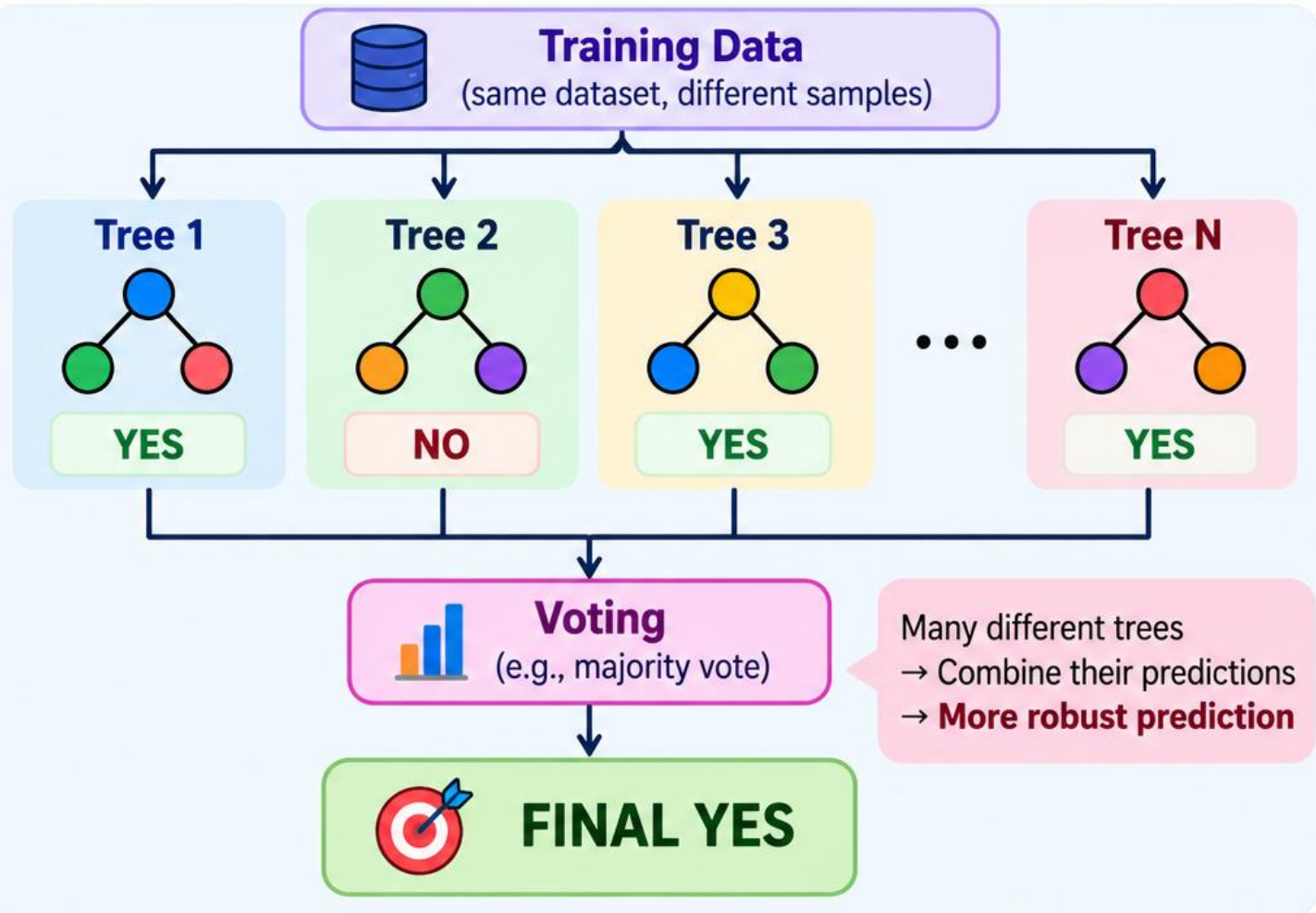
Each tree makes a prediction.

4

We combine all the predictions (**voting**).

5

This gives a more **robust and reliable prediction**.



Key Takeaway

“ Random Forest = Many Decision Trees working together to make a better and more reliable prediction. ”





How Does Random Forest Work?

It builds many decision trees using random samples and random features, and then combines their predictions.



1

Bootstrap Sampling

Create different training datasets by sampling with replacement.



2

Random Feature Selection

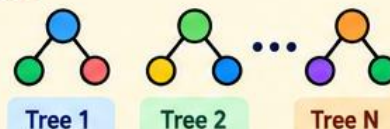
At each split, consider a random subset of features.

Features	
☒	Age
☐	Income
☒	Credit Score
☐	Employment
...	

3

Build Multiple Decision Trees

Build many Decision Trees, where each tree sees different samples and feature subsets.



4

Combine Predictions

Combine the predictions from all trees. The method depends on the problem type.

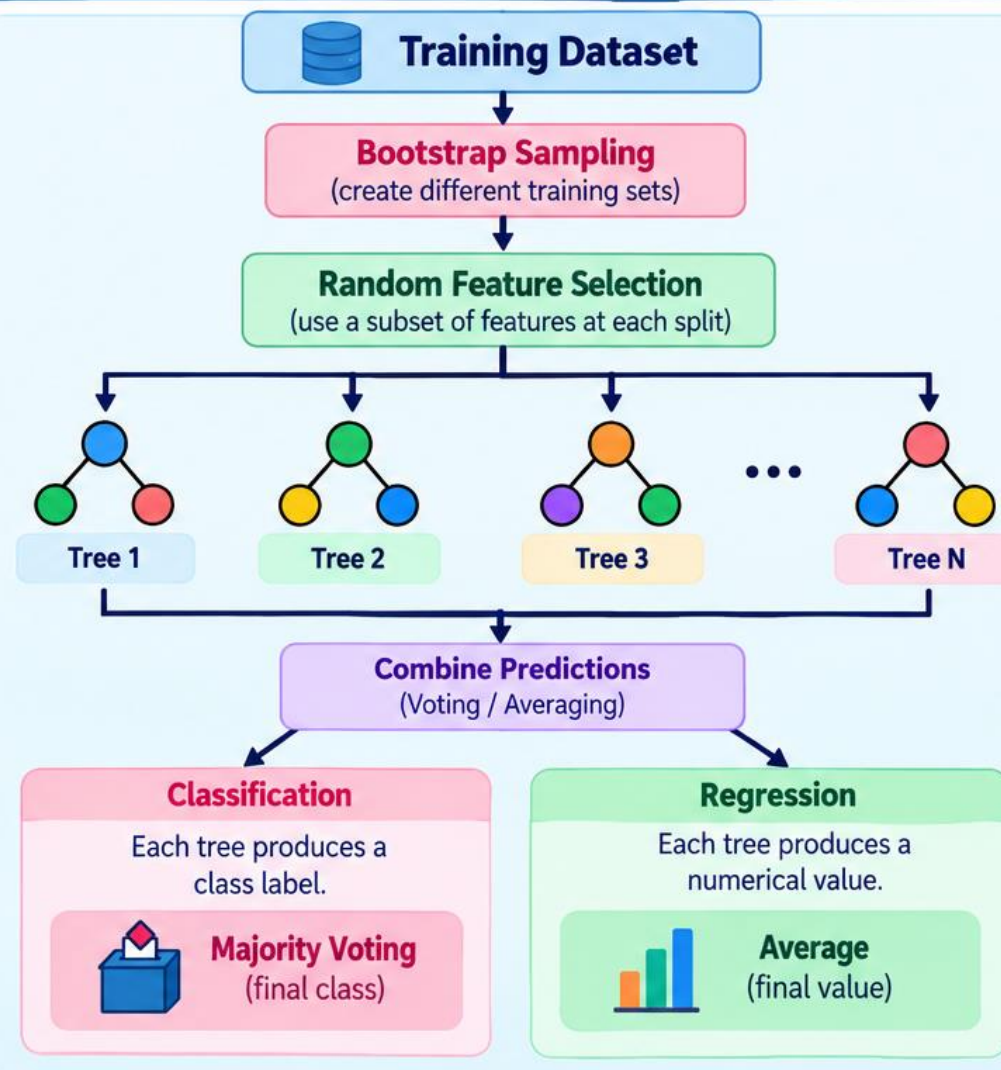
Voting / Averaging



Key Takeaway

Random Forest = Multiple Decision Trees + Randomness + Aggregation

The multiple trees are the heart of Random Forest!





Random Forest – Pros and Cons



A powerful model, but like every method, it has strengths and limitations.



Advantages (Pros)



Good prediction accuracy

Usually gives high and reliable performance.



Reduces overfitting

Combines many trees, so it is more stable than a single tree.



Works with different types of data

Handles both numerical and categorical features.



Works well on complex datasets

Can capture non-linear relationships.



Disadvantages (Cons)



Less interpretable

It is harder to understand compared to a single decision tree.



More computational cost

Building many trees takes more time and memory.



More parameters to tune

We need to choose settings like the number of trees, etc.



Key Takeaway

Random Forest gives strong and reliable predictions, but it is less interpretable and more computationally expensive than a single decision tree.



When to Use **Decision Tree** and **Random Forest**?

Both are useful – the right choice depends on your goal and data!

Use Decision Tree when...



You want a simple and easy-to-understand model

The tree can be visualized and explained easily.



Your dataset is small

Works well on small to medium-sized data.



Interpretability is important

Useful when you need to explain the decisions (e.g., in business or healthcare).



You need a quick model

Builds faster compared to Random Forest.



**Choose Decision Tree
when simplicity and
interpretability matter!**

Use Random Forest when...



You need higher prediction accuracy

Random Forest usually performs better than a single decision tree.



Your dataset is large or complex

Handles large datasets with many features well.



You want a more robust model

Less prone to overfitting (more stable and reliable).



Your data has noise or many features

Works well even when data is noisy or has non-linear relationships.



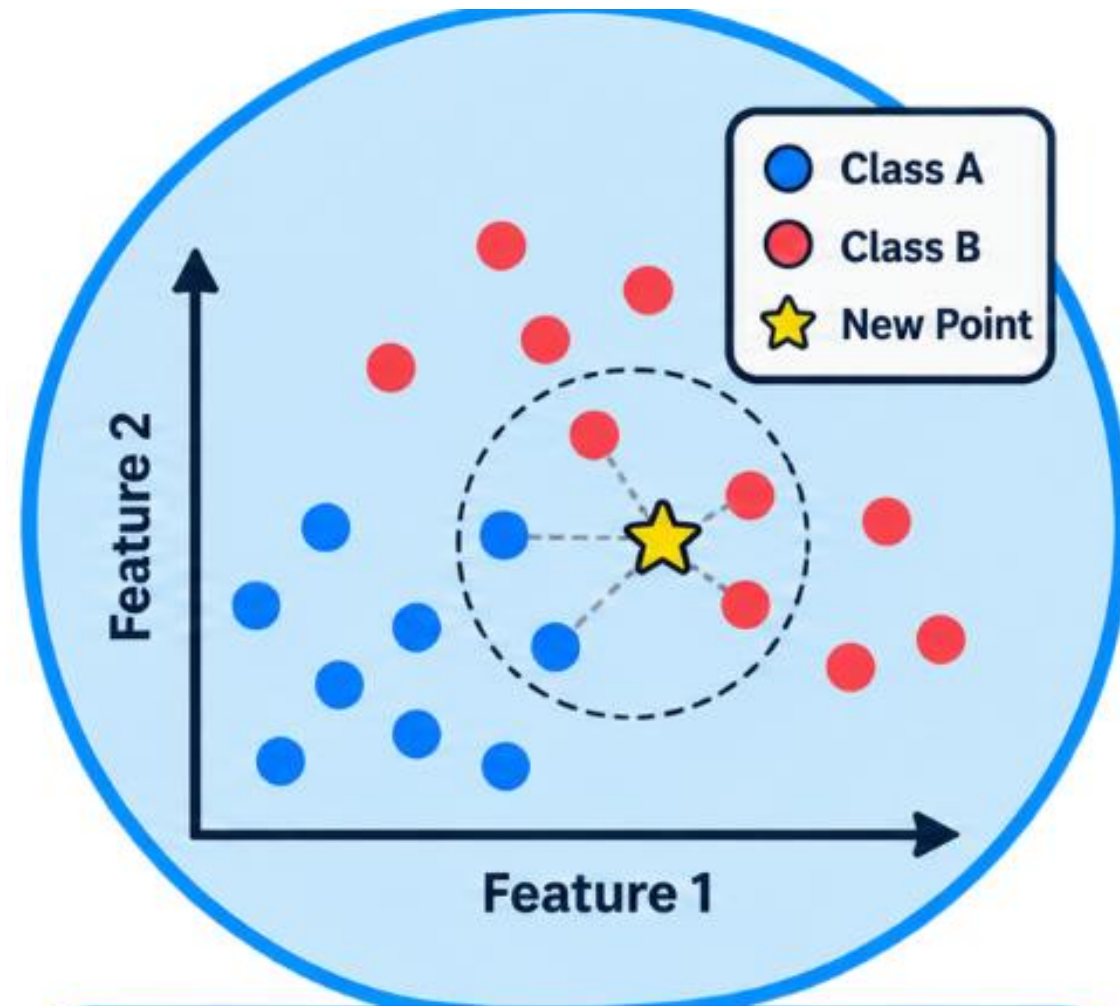
**Choose Random Forest
when you need better accuracy
and robustness!**



Key Takeaway

Use Decision Tree for simplicity and clear explanations.
Use Random Forest for higher accuracy and stronger performance.





K-Nearest Neighbors (KNN)



K-Nearest Neighbors (KNN)

A Simple Classification Algorithm

Similar data points belong to the same class!

What is KNN?



KNN is a **Supervised Learning** algorithm that classifies a new data point by looking at its **nearest neighbors**.

Core Idea

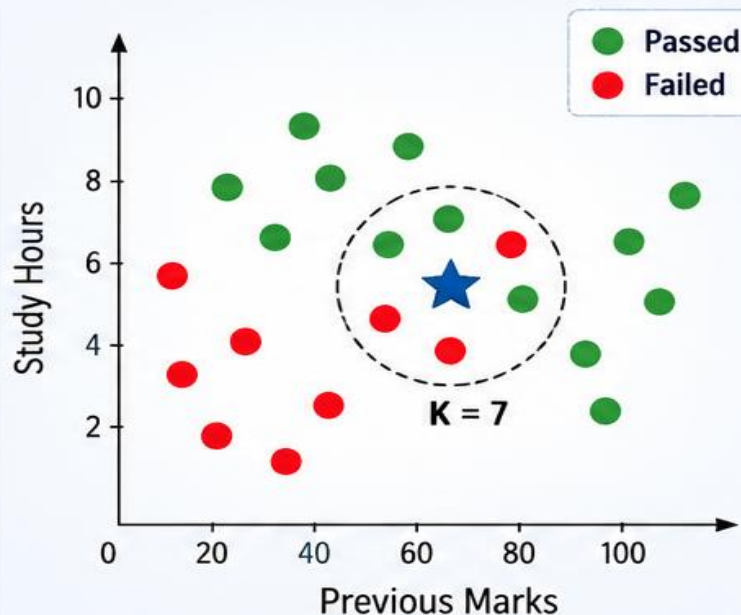
“Similar data points usually belong to the same class.”



Key Points to Remember

- **K** = Number of Neighbors
- **Nearest** = Most Similar Points
- **Majority Vote** = Prediction

Example: Student Pass/Fail



How it works?

- 1 Choose a value for K (number of neighbors).
- 2 Find the K nearest data points to the new point.
- 3 Look at the class labels of these neighbors.
- 4 Assign the class that has the majority (most common) label.

Example

A new student is similar to:

- 5 students who Passed
- 2 students who Failed

Majority = PASS

Prediction: PASS ✓

Example with K = 7

Among the 7 nearest neighbors:

- 4 belong to Class B
- 3 belong to Class A

Class B wins 4 – 3 → The point is Class B.



One-line Explanation

“KNN looks at the K closest data points and assigns the new point to the class that has the majority.”

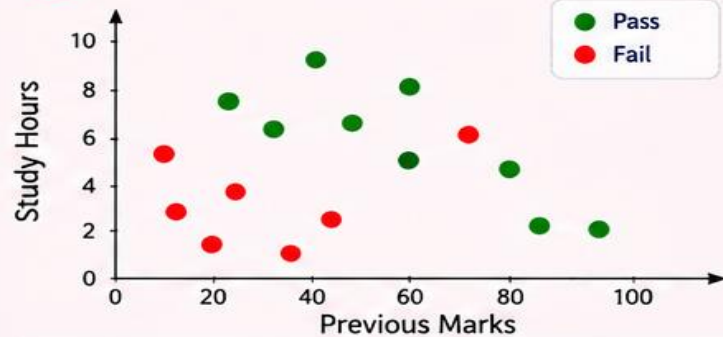


K-Nearest Neighbors (KNN) – Step by Step Example

A simple way to classify a new data point based on its nearest neighbors

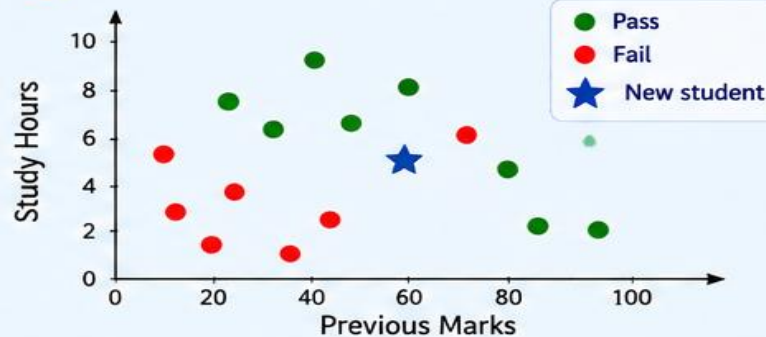
Similar points belong to the same class!

1 Plot the existing data



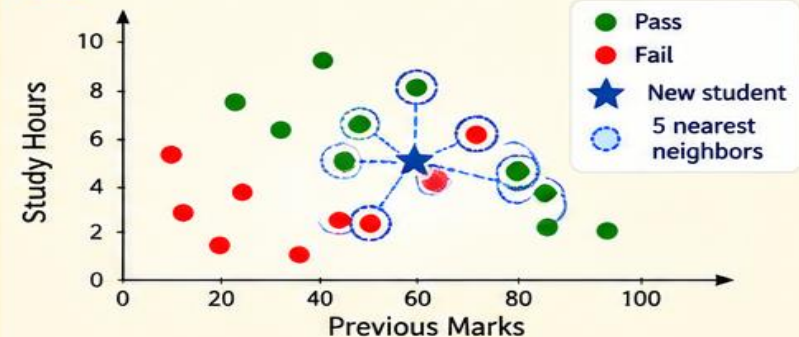
Each point represents a student. Its position is based on two features: Previous Marks and Study Hours.

2 Add the new data point



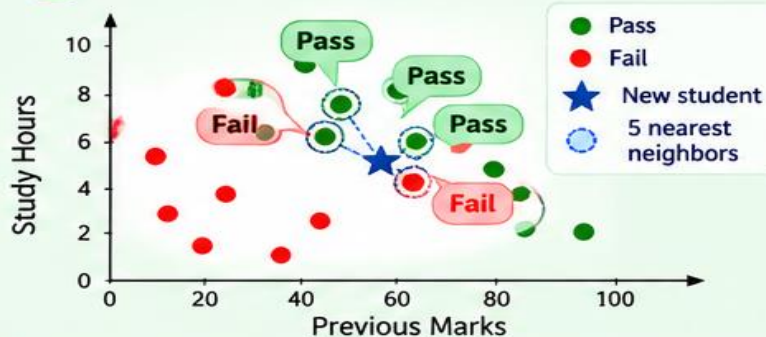
The blue star (★) is a new student. We don't know if this student will Pass or Fail.

3 Find the nearest neighbors (K = 5)



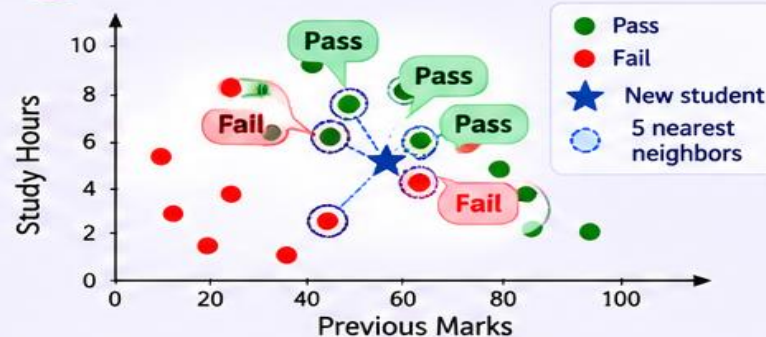
We find the 5 closest students to the new student (using a distance measure, e.g., Euclidean distance).

4 Look at their classes



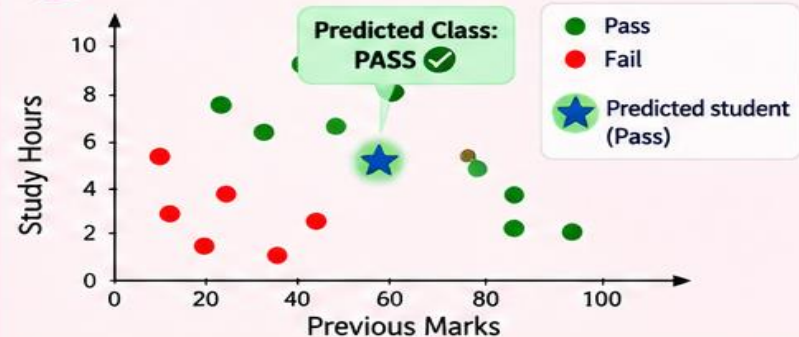
Among the 5 nearest neighbors: 3 students Passed (green) and 2 students Failed (red).

5 Majority wins



Since the majority of the 5 nearest neighbors are from the Pass class (3 out of 5), we assign the new student to the Pass class.

6 Final prediction



The new student is classified as PASS because most of its nearest neighbors passed.



Key Takeaway:

KNN looks at the **K closest** (most similar) data points and assigns the new point to the class that has the **majority**.



Simple idea.
Powerful in practice!



From data
to decision
in 5 steps!

How Does **KNN** Work?

KNN in
5 Simple Steps

1 Choose K

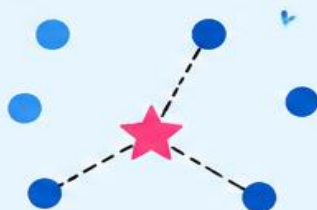
How many
neighbors should
we consider?

Example:
K = 3 or 5 or 7

K = ?

2 Calculate Distance

Find how close
the data points are
to the new point.

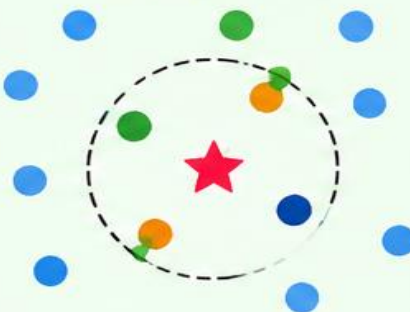


Common distance:
Euclidean Distance

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

3 Find K Nearest Neighbors

Select the K closest
data points.



★ New data point:
● Class A neighbor
● Class B neighbor
● Other data point

4 Check their Class Labels

Look at the class
labels of the K
nearest neighbors.

Example (K = 5):

● Class A
● Class A
● Class A
● Class B
● Class B

5 Majority Vote

The class with the
most neighbors
wins!

In this example:

● Class A = 3
● Class B = 2

**Predict the class
of the new data point.**



Final Output:

Assign the new data point to the class with the majority of its K nearest neighbors.

"KNN uses the K closest data points and assigns the new point to the class that has the majority."

Simple idea.
Powerful results!



KNN – Learn It With 6 Data Points

Example: Will the new student X Pass or Fail?

Same steps as before,
now with a real example!

1 Training Data (6 students)

We have 6 students with known results (Pass/Fail) based on:

- Hours studied
- Attendance (%)

Student	Study Hours (x)	Attendance (%) (y)	Result
A	1	50	Fail
B	2	55	Fail
C	3	60	Fail
D	6	80	Pass
E	7	85	Pass
F	8	90	Pass

New student X:

X	6.5	82	?
---	-----	----	---

Will X Pass or Fail?

2 Choose K

Let's choose

K = 3

We will look at the 3 closest students to X.

3 Calculate Distance

Use Euclidean distance:

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

Student	Calculation (from X = (6.5, 82))	Distance
A	$\sqrt{(5.5^2 + 32^2)}$	32.47
B	$\sqrt{(4.5^2 + 27^2)}$	27.37
C	$\sqrt{(3.5^2 + 22^2)}$	22.28
D	$\sqrt{(0.5^2 + 2^2)}$	2.06
E	$\sqrt{(0.5^2 + 3^2)}$	3.04
F	$\sqrt{(1.5^2 + 8^2)}$	8.14

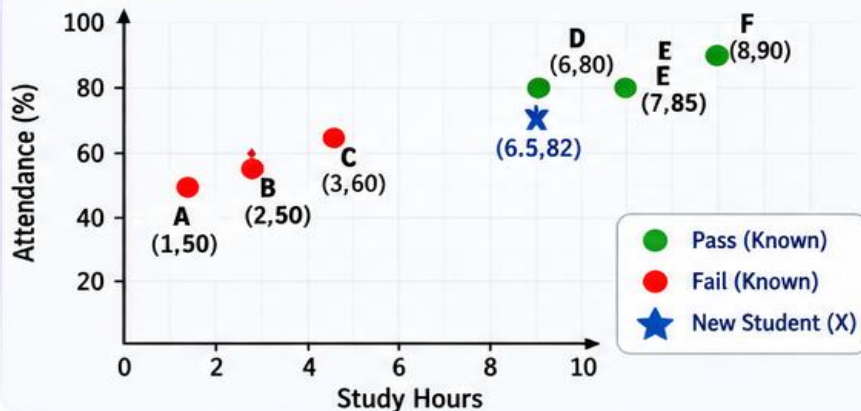
4 Find K Nearest Neighbors

Sort the distances (smallest first):

Rank	Student	Distance	Result
1st	D	2.06	Pass
2nd	E	3.04	Pass
3rd	F	8.14	Pass
4th	C	22.28	Fail
5th	B	27.37	Fail
6th	A	32.47	Fail

The 3 nearest neighbors are:
D, E, F

Data Points on a Graph



5 Check Class Labels & Majority Vote

Look at the class labels of the 3 nearest neighbors (D, E, F).

Student	Distance	Result
D	2.06	Pass
E	3.04	Pass
F	8.14	Pass

Pass = 3
Fail = 0

Prediction:

PASS ✓



Key Takeaways

- ✓ KNN finds the K closest data points based on distance.
- ✓ It does not know the neighbors beforehand — it discovers them by calculating distances.
- ✓ Classification is done by majority vote.
- ✓ In this example, the new student X is classified as **PASS**.

Simple data.
Clear steps.
Easy to understand!



Final Answer: The new student X (6.5, 82) is predicted as **PASS** because most of its 3 nearest neighbors (D, E, F) are Pass.



Important Points in KNN



What Should We Remember?

Small things
make a big
difference!

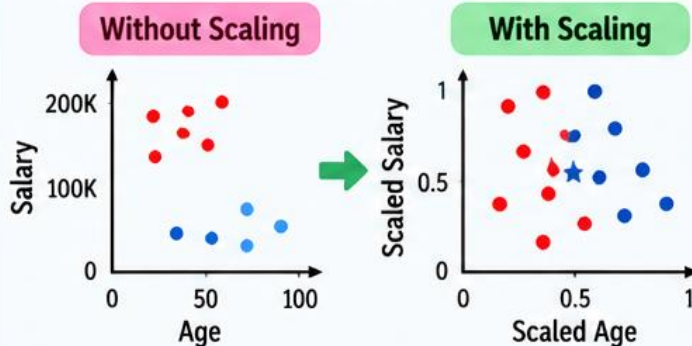
1 Feature Scaling is Important

KNN uses distance, so features should be on **comparable scales**.

Example:

Feature	Typical Range
Age	18 – 60
Salary	₹20,000 – ₹2,00,000

➔ Scaling prevents one feature from **dominating** the distance.



2 Choosing K Matters

Small K → **Sensitive to noise**

- Decision can change easily due to a few points.
- May overfit the data.



K = 1
(more sensitive)

Large K → **May become too general**

- Considers many neighbors.
- May ignore local patterns.



K = 7
(more general)



Choose a suitable K

(using experiment or cross-validation).

3 KNN is a Lazy Learner

- KNN does not build a model during training.
- It mainly **stores the training data**.
- It performs distance calculations only **when a new data point needs to be predicted**.

Training Phase



Store the training data
(no model building)

Prediction Phase



Calculate distances and find K nearest neighbors for the new data point

So, KNN is called a **“Lazy Learner”** because it waits until prediction time to do the work.



Key Takeaway:

Scale your data, choose K carefully, and remember that KNN is a lazy learner.

Simple ideas
lead to better
predictions!

Let's look at the good and the not-so-good!

KNN: Pros and Cons

Every algorithm has strengths and weaknesses!



Understand both sides to use KNN better!



Advantages of KNN



Simple to understand

The idea is easy and intuitive – classify based on nearest neighbors.



Easy to implement

Requires only a few steps and no complex training process.



Good for small datasets

Works well when the dataset is not very large.



Works for multi-class classification

Can handle two or more classes easily.



Limitations of KNN



Can be slow for large datasets

It needs to calculate the distance to every training point, which can be time-consuming for big data.



Sensitive to feature scaling

Features should be on similar scales, otherwise features with larger values may dominate the distance.



Sensitive to irrelevant features

Irrelevant or noisy features can affect the distance calculation and lead to poor predictions.



Choosing K is important

A small K can be sensitive to noise, while a large K may be too general.

Final Takeaway

KNN is a **simple and effective** algorithm, especially for small datasets, but it requires **careful preprocessing** and the **right choice of K** for good results.

Use KNN wisely for better predictions!

Applications of Machine Learning



Real-World Applications

Same ideas. Different uses. Real impact!

Machine Learning helps solve real problems around us!



Decision Tree (DT)

Simple rules for decision making

DT splits the data step by step using simple if-else rules, making it easy to understand and explain.

Real-World Applications



Healthcare

Predicting whether a patient has a disease (e.g., diabetes: Yes/No).



Loan Approval

Deciding whether to approve or reject a loan application.



Student Performance

Classifying students as Pass/Fail based on study habits, attendance, etc.



Customer Churn

Predicting whether a customer will leave a service (e.g., telecom, banking).



Random Forest (RF)

Stronger predictions using many trees

RF combines the predictions of many decision trees, giving more accurate and stable results.

Real-World Applications



Fraud Detection

Identifying fraudulent transactions in banking and online payments.



Healthcare

Predicting risk of diseases (e.g., heart disease, diabetes).



Predictive Maintenance

Predicting equipment or machine failure in industries.



Demand Forecasting

Predicting product demand in retail and supply chain.



K-Nearest Neighbors (KNN)

Find similar cases, make a prediction

KNN classifies a new data point based on the most similar (nearest) examples from past data.

Real-World Applications



Recommendation Systems

Suggesting similar movies, products or music based on user preferences.



Healthcare

Finding similar patients to predict disease outcome or suggest treatment.



Spam Detection

Classifying emails as spam or not spam by comparing with similar past emails.



Location-Based Services

Finding nearby places (restaurants, hotels, ATMs) based on user location.

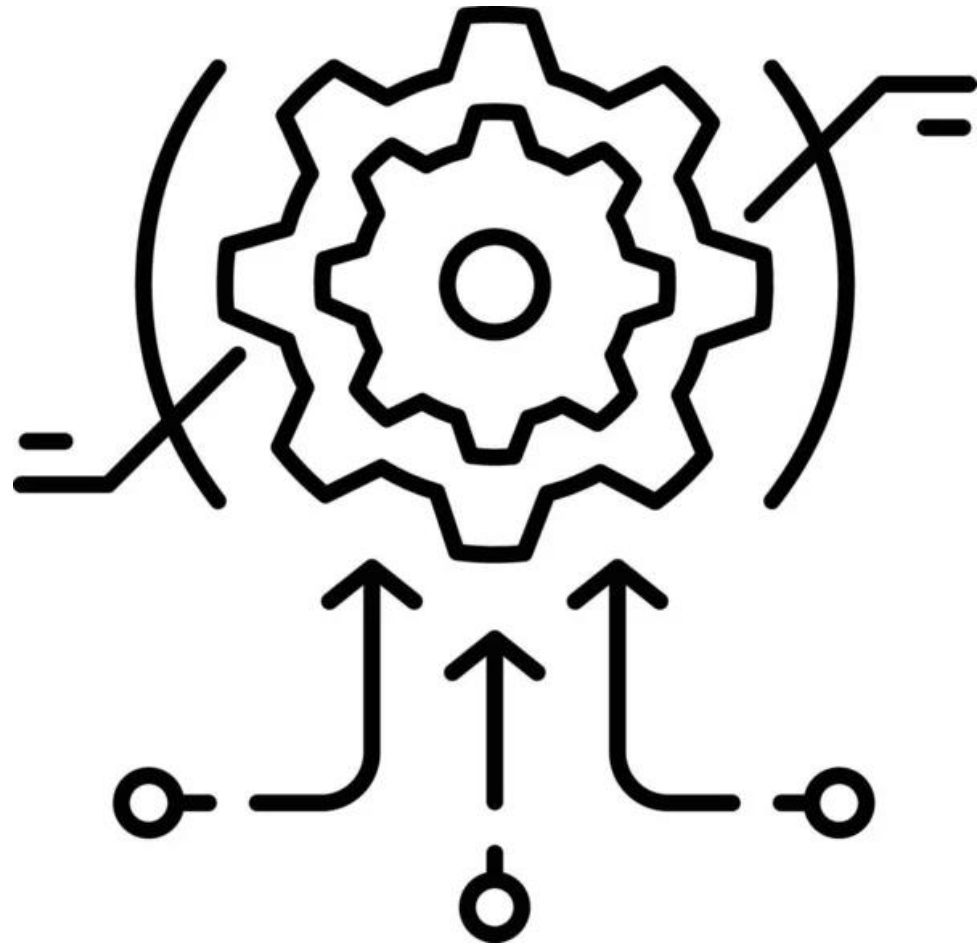


Key Takeaway:

DT, RF and KNN are powerful tools that can be applied to solve many **real-world problems** across different domains!

Different algorithms. A smarter future!

Performance Evaluation of ML Algorithms





Actual vs Predicted Value, Accuracy and Error Rate

Let's see how well our model is doing! ➡

Compare what really happened with what the model predicted!

1 Actual Value

The **true** or **correct** answer in the real world.



Example:
A student **actually Passed**.

2 Predicted Value

The answer given by the **machine learning model**.



Example:
The model predicted **Passed**.

3 Comparison

Correct Prediction

Actual: **Pass**
Predicted: **Pass**



Actual = Predicted
→ **Correct**

Incorrect Prediction (Error)

Actual: **Fail**
Predicted: **Pass**



Actual ≠ Predicted
→ **Error**

4 Example: 10 Students

Student	Actual	Predicted	Result
1	Pass	Pass	✓
2	Pass	Pass	✓
3	Fail	Fail	✓
4	Pass	Fail	✗
5	Fail	Fail	✓
6	Pass	Pass	✓
7	Fail	Pass	✗
8	Pass	Pass	✓
9	Fail	Fail	✓
10	Pass	Pass	✓

5 Accuracy

How often is the model **correct**?

$$\text{Accuracy} = \frac{\text{Number of Correct Predictions}}{\text{Total Predictions}}$$

In our example:

- Correct predictions = 8
- Total predictions = 10

$$\text{Accuracy} = \frac{8}{10} = 0.80 = \mathbf{80\%}$$

6 Error Rate

How often is the model **wrong**?

$$\text{Error Rate} = \frac{\text{Number of Incorrect Predictions}}{\text{Total Predictions}}$$

In our example:

- Incorrect predictions = 2
- Total predictions = 10

$$\text{Error Rate} = \frac{2}{10} = 0.20 = \mathbf{20\%}$$



Key Takeaways

- **Actual value** = what really happened.
- **Predicted value** = what the model said.
- **Accuracy** = percentage of correct predictions.
- **Error rate** = percentage of incorrect predictions.

$$\text{Accuracy} + \text{Error Rate} = 100\%$$



Performance Evaluation in Machine Learning



We use different metrics to check how well our model is performing, depending on the type of problem.

1 Classification Metrics

Used when the target/output is a **class or category**.

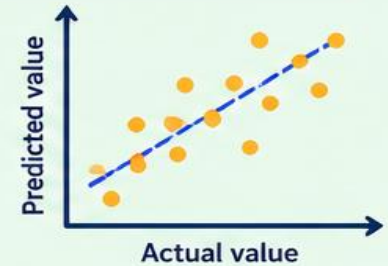


Main metrics:

	Confusion Matrix	Shows correct and incorrect predictions for each class.
	Accuracy	Overall proportion of correct predictions.
	Precision	Of the predicted positives, how many are actually correct?
	Recall / Sensitivity	Of the actual positives, how many did we correctly identify?
	F1-Score	Harmonic mean of Precision and Recall (balances both).

2 Regression Metrics

Used when the target/output is a **continuous numerical value**.



Main metrics:

	MAE Mean Absolute Error	Average of the absolute differences between actual and predicted values.
	MSE Mean Squared Error	Average of the squared differences between actual and predicted values.
	RMSE Root Mean Squared Error	Square root of MSE. Gives error in the same units as the target variable.



Choose the right metrics for your problem to understand how well your model is performing!



Confusion Matrix

Compare Actual vs Predicted Results

A simple table that shows how many predictions are correct and incorrect!



What is a Confusion Matrix?

A confusion matrix is a table that compares the **actual** (true) class with the **predicted** class given by the model.

Confusion Matrix for Binary Classification

		Actual (True Class)	
		Pass (Positive)	Fail (Negative)
Predicted (by Model)	Pass (Positive)	TP (True Positive) ✓ Actually Pass → Predicted Pass	FP (False Positive) ✗ Actually Fail → Predicted Pass
	Fail (Negative)	FN (False Negative) ✗ Actually Pass → Predicted Fail	TN (True Negative) ✓ Actually Fail → Predicted Fail

Meaning of Each Term

TP

True Positive

Actually Pass → Predicted Pass
(Correct prediction)

FP

False Positive

Actually Fail → Predicted Pass
(Wrong prediction / false alarm)

FN

False Negative

Actually Pass → Predicted Fail
(Wrong prediction / missed case)

TN

True Negative

Actually Fail → Predicted Fail
(Correct prediction)



Goal



High values on the **diagonal (TP and TN)**.



Low values on the **off-diagonal (FP and FN)**.



This means fewer mistakes and **better model performance**.

The diagonal values (TP and TN) **should be high** for a good model!



Accuracy, Precision and Recall

Let's understand these metrics using a real example (Decision Tree Confusion Matrix)

Simple
Concepts
Big Opportunities!

Confusion Matrix - Decision Tree

		Predicted Class		
		1	2	3
Actual Class	1	12	0	0
	2	0	14	0
	3	0	1	9



Accuracy

Overall, how many predictions are correct?

$$\text{Accuracy} = \frac{\text{Total correct predictions}}{\text{Total predictions}}$$



Precision

Of the items predicted as a class, how many are actually that class?

$$\text{Precision} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Positives}}$$



Recall

Of the actual items in a class, how many did we correctly predict?

$$\text{Recall} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Negatives}}$$

Precision and Recall for each class (one-vs-all)

Class 1

TP = 12 FP = 0 FN = 0

Precision = $12 / (12 + 0) = 1.00$

Recall = $12 / (12 + 0) = 1.00$

Class 2

TP = 14 FP = 1 FN = 0

Precision = $14 / (14 + 1) = 0.93$

Recall = $14 / (14 + 0) = 1.00$

Class 3

TP = 9 FP = 0 FN = 1

Precision = $9 / (9 + 0) = 1.00$

Recall = $9 / (9 + 1) = 0.90$

Overall Accuracy

Total correct predictions = $12 + 14 + 9 = 35$

Total predictions = 36

$$\text{Accuracy} = \frac{35}{36} = 0.9722 \quad (97.22\%)$$



Higher **accuracy**, **precision**, and **recall** mean a better performing model – and these are key skills you should know for interviews and real-world projects!

Learn Metrics
Build Confidence
Get Job Ready!



Confusion Matrix

A confusion matrix compares the actual (original) and predicted values to see how many predictions are correct and incorrect.

Simple
but Powerful!

		Predicted Value	
		True (Positive)	False (Negative)
Actual Value	True (Positive)	True Positive (TP) ✓	False Negative (FN) ✗
	False (Negative)	False Positive (FP) ✗	True Negative (TN) ✓



True Positive (TP)

Both the original data points and the predicted data points are true.

Example:

Actual: Has disease
Predicted: Has disease



True Negative (TN)

Both the original data points and the predicted data points are false.

Example:

Actual: No disease
Predicted: No disease



False Positive (FP)

Original data points are false, but the predicted data points are true.

Example:

Actual: No disease
Predicted: Has disease



False Negative (FN)

Original data points are true but the predicted data points are false.

Example:

Actual: Has disease
Predicted: No disease



A confusion matrix helps us understand what our model gets right and what it gets wrong.

Better
Models
for a Brighter
Tomorrow!



Precision – What does it actually mean?

When the model says YES, how often is it actually YES? 

Precision tells us how trustworthy our positive predictions are.

1 Our Example: Student Pass/Fail

Positive class
= **Pass**

Negative class
= **Fail**



Pass
(Positive)



Fail
(Negative)

2 Model Prediction

The model predicts:
“These 10 students
will **PASS**.”



Predicted
Pass for
10 students

3 What actually happened?

After checking the actual results:



7 students
actually **passed**.



3 students
actually **failed**.



So, the model said **PASS** for **10 students**,
but only **7** were really **Pass**.

4 Precision Formula

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

$$\text{Precision} = \frac{7}{7 + 3} = \frac{7}{10} = \mathbf{70\%}$$

5 In Plain English



“Of all the students that
the model **predicted as Pass**,
how many **actually passed**?”

Answer: **70%**

Where do TP and FP come from?

		Actual (True Class)	
		Pass (Positive)	Fail (Negative)
Predicted (by Model)	Pass (Positive)	TP (7)	FP (3)
	Fail (Negative)	FN	TN



Higher Precision → When the model says PASS, it is usually correct!





Recall – What does it actually mean?

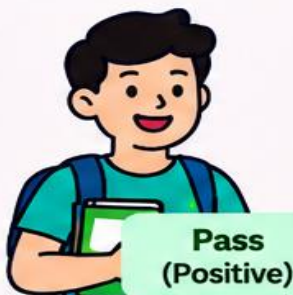
Among all the Actual Positive cases, how many were correctly identified as Positive by the model?

Recall tells us how many of the actual positive cases we were able to find!

1 Our Example: Student Pass/Fail

Positive class
= **Pass**

Negative class
= **Fail**



Pass
(Positive)



Fail
(Negative)

2 What are we looking at?

Recall looks at all the **Actual Positive cases** (the students who really passed).



Out of all the students who **actually passed**, how many did the model correctly identify as Pass?

3 Confusion Matrix (Focus on Actual Positive)

		Actual (True Class)	
		Pass (Positive)	Fail (Negative)
Predicted (by Model)	Pass (Positive)	TP	FP
	Fail (Negative)	FN	TN

Recall looks at this column (all actual positives).

4 Example

Suppose there are **10 students** who **actually passed** (Actual Positive).

The model correctly identifies:



8 students as **Pass (TP)**



2 students as **Fail (FN)**

5 Recall Formula and Calculation

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

$$\text{Recall} = \frac{8}{8 + 2} = \frac{8}{10} = \mathbf{80\%}$$

6 In Plain English



“Among all the students who **actually passed**, how many did the model correctly identify as **Pass**?”

Answer: **80%**



Higher Recall → The model is better at finding the actual positive cases.

Recall focuses on finding the positives!

RoadMap and Outcome of the Session





Real-World Problems with Machine Learning

From Data to Real Impact!

Same Algorithms,
Bigger Solutions!

Real-World Problem		ML Algorithm / ML Problem	SIH Relevant
	Groundwater Level Prediction	Regression — Random Forest, XGBoost, SVR	SIH1696 – Ground Water Level Predictor
	Agricultural Commodity Price Prediction	Regression / Forecasting — Random Forest, XGBoost, Linear Regression	SIH1647 – AI-ML based commodity price prediction
	Reservoir Water Requirement Prediction	Regression / Forecasting — Random Forest, XGBoost, SVR	SIH1692
	Traffic Congestion Prediction	Classification / Regression — Random Forest, XGBoost, SVM	SIH1607
	Truck Route / Travel-Time Prediction	Regression — Random Forest, XGBoost, Linear Regression	Road Transport Network Telematics
	Crop Yield Prediction	Regression — Random Forest, XGBoost, SVR	SIH1639 – Sustainable Fertilizer Usage Optimizer
	Water Quality Prediction	Classification — Random Forest, SVM, Decision Tree	SIH water-resource / water-quality challenges
	Industrial Machine Failure Prediction	Classification — Random Forest, XGBoost, SVM	SIH-inspired
	Flood Risk Prediction	Classification — Random Forest, XGBoost, Decision Tree	SIH-inspired
	Fertilizer Recommendation	Classification / Recommendation — Decision Tree, Random Forest, XGBoost	SIH1639



Real problems. Real data. Real impact.

Machine Learning can help build a better, smarter, and sustainable tomorrow!



Ideas today
for a brighter
tomorrow!



Major **ML/DL** Dataset Repositories

Find. Download. Learn. Build. Innovate!

Great datasets
lead to
great solutions!

1



IEEE DataPort

<https://ieee-dataport.org/>

2



Mendeley

Mendeley Data

<https://data.mendeley.com/>

3



Machine Learning Repository

UCI Machine
Learning Repository

<https://archive.ics.uci.edu/>

4



Kaggle Datasets

[https://www.kaggle.com/
datasets](https://www.kaggle.com/datasets)

5



OpenML

OpenML

[https://www.openml.org/
search?type=data](https://www.openml.org/search?type=data)

6



Zenodo

<https://zenodo.org/>

7



figshare

Figshare

<https://figshare.com/>

8



PhysioNet

<https://physionet.org/>

9



Microsoft
Research

Microsoft Research
Open Data

[https://www.microsoft.com/
research/open-data](https://www.microsoft.com/research/open-data)

10



Dataset Search

Google Dataset Search

[https://datasetsearch.
research.google.com/](https://datasetsearch.research.google.com/)



Key Takeaway:

A **good dataset** is the first step towards
a **successful Machine Learning project!**

Explore datasets
Build skills
Create impact!

Major ML/DL Dataset Repositories

- **IEEE DataPort** — Research-oriented datasets, particularly useful for ML, AI, signal processing, IoT, computer vision, healthcare, etc. IEEE DataPort currently lists thousands of datasets across AI and ML categories.
[IEEE DataPort – Dataset Repository](#)
- **Mendeley Data** — Excellent source for **research datasets**, including ML, DL, computer vision, medical imaging, agriculture, cybersecurity, NLP, etc. It provides DOI-based datasets and supports versioning.
[Mendeley Data](#)
- **UCI Machine Learning Repository** — One of the **best sources for beginners learning classical ML**. It contains datasets such as Iris, Heart Disease, Wine Quality, Bank Marketing, etc.
[UCI Machine Learning Repository](#)
- **Kaggle Datasets** — Very popular for **ML/DL practice, competitions, projects, and teaching**. It contains tabular, image, text, time-series and many other datasets.
[Kaggle Datasets](#)
- **OpenML** — Particularly useful for **machine-learning experiments and benchmarking**. It provides thousands of datasets and allows datasets to be downloaded through its website or API. [OpenML Dataset Repository](#)
- **Zenodo** — A general research repository where researchers publish datasets, software, papers, and supplementary research material. It is particularly useful when you need a **citable dataset with a DOI**.
[Zenodo](#)
- **Figshare** — Research data repository useful for datasets, figures, code, and supplementary research materials. IEEE also recommends Figshare among alternative research data repositories.
[Figshare](#)
- **PhysioNet** — Excellent for **healthcare, biomedical, ECG, EEG, physiological signals and medical datasets**. Some datasets are open access while others require credentialed access.
[PhysioNet](#)
- **Microsoft Research Open Data** — Useful for research datasets covering computer vision, NLP, healthcare, geospatial data and other areas.
Microsoft Research Open Data
- **Google Dataset Search** — Not a repository itself, but a **dataset search engine** that helps you discover datasets hosted by universities, research repositories and organizations.
[Google Dataset Search](#)



Popular Tools for Machine Learning Experimentation

Run Code • Experiment • Learn • Build • Innovate

Choose the tool that fits your goals!

Free (Open Source) Tools

Great for learning, practice and academic projects!



1 Google Colab



Cloud-based platform to run Python code in the browser. No installation required.

Best for:
Beginners, quick experiments, no setup.

2 Jupyter Notebook



Interactive notebook environment for running Python code locally.

Best for:
Learning, data analysis, local experimentation.

3 Kaggle Notebook



Free cloud notebooks with access to many public datasets and competitions.

Best for:
Dataset exploration, competitions, practice.

4 VS Code



Lightweight and powerful code editor for ML project development.

Best for:
Coding, project development, extensions support.

5 Anaconda



Open-source Python distribution for managing environments and packages.

Best for:
Managing Python environments for ML projects.

Paid Tools (Subscription / Credits)

Useful for advanced experimentation, larger datasets and professional projects!



6 Paperspace



Cloud platform with GPU instances for ML/DL experimentation.

Best for:
Deep learning, GPU-based experiments.

7 Gradient



Cloud platform for ML/DL with ready-to-use environments and GPUs.

Best for:
Scalable experimentation, team projects.

8 Deepnote



Collaborative cloud notebooks for data science and ML.

Best for:
Team collaboration, sharing work easily.

9 IBM Watson Studio



Cloud platform for data science and ML model development.

Best for:
Enterprise-level projects, collaboration.

10 AWS SageMaker



Fully managed platform for building, training and deploying ML models.

Best for:
Advanced ML projects, real-world deployment.



Different tools, same goal – Learn, Experiment, Create Impact!

“The best tool is the one that helps you learn and build consistently.”



Start Small
Think Big
Keep Learning!



After This Session: Your ML Journey Starts!

Learn • Practice • Explore • Create • Innovate • Make an Impact!

Same Curiosity
Bigger Possibilities!

1 What You Have Learned Today

- ✓ A complete overview of ML foundations
- ✓ How ML problems are defined and solved
- ✓ Types of ML and important algorithms
- ✓ Data preprocessing, feature scaling and feature engineering
- ✓ How to evaluate and compare ML models
- ✓ How to think like an ML practitioner and researcher



Your foundation is ready!

2 Now — Kick-Start Your Own ML Project

1 Choose a Real-World Problem

Start with a problem that interests you.



Find a problem that excites you!

2 Find the Data

Explore datasets from **Kaggle, UCI, IEEE DataPort, Mendeley Data, Zenodo**, etc.



Good data leads to good models!

3 Start with Reverse Engineering

- Find an existing ML project, paper or solution
- Understand what problem it solves
- Study the dataset, features, algorithm and evaluation
- Reproduce the basic approach
- Experiment with different models or features
- Identify what can be improved

4 Move from Reproduction to Innovation

Existing Solution

↓
Understand

↓
Reproduce

↓
Experiment

↓
Find Limitation

↓
Improve

↓
Your Own Solution

3 How to Become an ML Expert

A Continuous Learning Journey



Learn

Build strong foundations



Practice

Solve problems regularly



Build Projects

Work on real-world data



Compete

Join Kaggle, SIH and more



Read Papers

Explore latest research



Reproduce

Implement existing work



Experiment

Try new ideas and compare



Research

Find gaps and propose solutions



Publish

Share your work with the world

You Can Do It!



Final Message

“You don’t need to know everything before starting your first ML project. **Start with an existing solution, understand it deeply, experiment with it, and gradually build your own.**”

Your ML Journey Has Just Begun!

Same Learner
Stronger Tomorrow!



Best Practices for Becoming an ML Researcher

Learn Continuously • Build • Explore • Contribute • Make an Impact!

Small Steps Today
Big Discoveries Tomorrow!

Practice		What to Do	
1	 Participate in Kaggle	Solve datasets and competitions; learn from other solutions.	
2	 Join ML Competitions	Participate in Kaggle , DrivenData , Aicrowd , etc.	
3	 Build Real-World Projects	Solve problems using real datasets rather than only tutorial datasets.	
4	 Read Research Papers	Regularly read papers from IEEE , Elsevier , Springer , ACM .	
5	 Reproduce Existing Work	Implement published ML methods and try to reproduce their results.	
6	 Find Research Gaps	Identify limitations in existing methods and propose improvements.	
		What to Do	
7	 Experiment with Multiple Models	Compare algorithms, features, preprocessing methods and hyperparameters.	
8	 Work with Different Datasets	Use UCI , Kaggle , IEEE DataPort , Mendeley Data , Zenodo , etc.	
9	 Participate in Hackathons	Join Smart India Hackathon (SIH) and other innovation challenges.	
10	 Publish Research	Convert strong experimental work into conference/journal papers .	
11	 Collaborate	Work with faculty, researchers and students from different domains.	
12	 Keep Learning	Follow new ML techniques, research trends, conferences and workshops.	



Curiosity + Practice + Persistence = Research Success!

"Be a learner today, a contributor tomorrow, and a researcher for a better future!"



Explore
Learn
Innovate
Make an Impact!



You Have the ML Foundation — What's Next?

Here's what you have learned today and how it prepares you for the next step!



ML Foundations

- What is Machine Learning?
- How ML problems are defined
- How they are solved

You now understand the basics.



ML Types & Algorithms

- Classification
- Regression
- Clustering
- Decision Tree (DT)
- Random Forest (RF)
- K-Nearest Neighbors (KNN)

You explored the main types and key algorithms.



Data Preparation

- Data cleaning
- Data transformation
- Feature scaling
- Feature engineering
- Feature selection

You learned how to prepare data for better models.



Model Evaluation

- Train / Validation / Test
- Evaluation metrics
- Compare models
- Understand performance

You know how to evaluate and choose the right model.



ML Mindset

- Think like an ML practitioner
- Ask the right questions
- Learn from examples
- Keep experimenting
- Be curious and explore

You are ready to apply what you learned!



Today you learned the foundation.
Now it is time to use it.



PRACTICE
EXPLORE
BUILD
GROW



Now — Start Your Own ML Project

Ideas
Create
Impact!

Apply what you learned. Explore. Experiment. Build your own.

1 Choose a Problem



Pick a real-world problem that interests you.

- Look around you
- Solve a real need
- Choose something you are curious about

"A good problem leads to better learning!"

2 Find the Data



Look for available datasets in trusted sources.

- Kaggle
- UCI Machine Learning Repository
- IEEE DataPort
- Mendeley Data
- Zenodo
- Government / NGO data
- Create your own dataset

"Good data is the foundation of ML."

3 Understand an Existing Solution



Study how others have solved a similar problem.

- Understand the problem
- Explore the dataset
- Check the features
- See which algorithm they used
- Learn how they evaluated it

"Learn from existing work. It saves time and gives you a strong start."

4 Reproduce, Experiment and Improve



- Reproduce the results from the existing solution
- Try different algorithms
- Experiment with different features
- Adjust parameters
- Compare the performance
- See what works better

"Experiment and explore. Small changes can lead to big improvements!"

5 Document and Share



- Write a short report or notebook
- Present your results
- Share on GitHub
- Create a simple presentation
- Get feedback
- Keep improving

"Share your work. It builds your confidence and helps others!"

"Don't feel that you have to invent a new algorithm for your first project."



Start with what already exists.

Understand it. Reproduce it.
Experiment with it. Improve it.



Real
Problems
Brighter
Futures

From Existing Work → Your Own Research

Start Small
Think Big!

Learn from what already exists. Reproduce. Experiment. Improve. Make it your own.

1 Existing Solution



Find a research paper or an existing project on a similar problem.

"You don't have to invent a new algorithm. Start from existing work."

2 Understand



- Understand the problem
- Understand the dataset
- Understand the features
- Understand why they used that algorithm

"Read, learn, ask questions."

3 Reproduce



- Implement the existing method
- Use the same dataset and settings
- Try to get similar results

"Reproducing results gives you confidence and deeper understanding."

4 Experiment



- Try different models
- Use different features
- Tune parameters
- Compare performance
- Analyze the results

"Small experiments can lead to new insights."

5 Find a Limitation



- Identify weaknesses
- Look for cases where it fails
- Find what can be improved
- Think of new ideas

"A limitation is an opportunity for your contribution."

6 Your Own Solution



- Propose an improvement
- Implement and test it
- Show better results (or new insights)
- Write it as a project or research paper

"This is where your research begins!"

Examples of Experiments



Different Models

- Decision Tree
- Random Forest
- KNN
- etc.



Different Features

- All features
- Selected features
- Engineered features
- etc.



Different Parameters

- Change hyperparameters
- Try different settings
- Compare results
- etc.



Research begins when you start asking:
"Can I make this better?"

*Same problem.
New ideas.
Brighter future.*



*Today a learner.
Tomorrow a researcher!*

Data Engineer vs Data Analyst vs Data Scientist

Role	Main responsibility	Typical work	Common tools
Data Engineer	Collect, transform and store data	Data pipelines, ETL/ELT, databases, data warehouses	Python, SQL, Apache Spark, Airflow, Kafka, Snowflake, Databricks
Data Analyst	Understand and communicate data insights	Reports, dashboards, business analysis	Excel, SQL, Power BI, Tableau
Data Scientist ML Engineer / MLOps Engineer	Analyze data and build ML models	EDA, visualization, feature engineering, model creation, model evaluation	Python, Jupyter, Pandas, NumPy, Matplotlib, Seaborn, Scikit-learn, XGBoost, TensorFlow, PyTorch
	Deploy and maintain ML models	Model deployment, APIs, monitoring, scaling, automation	Python, FastAPI, Docker, Kubernetes, MLflow, AWS/Azure/GCP

How Does Machine Learning Learn to Classify Wine?

Hands on:

<https://colab.research.google.com/drive/1VNaZVy-VL64Q56C1RHdBALFzuaFjseiQ>

Dataset Description

Item	Description
Dataset	Wine Dataset (https://archive.ics.uci.edu/dataset/109/winedataset?utm_source=chatgpt.com)
Number of Classes	3 — Class 1, Class 2, Class 3
Target Variable	Class
Input Features	13 chemical properties of wine
Examples of Features	Alcohol, Malic Acid, Magnesium, Flavanoids, Proline, Color Intensity, etc.
Data Type	Numerical
Problem Type	Multiclass Classification
Goal	Predict the wine class based on its chemical properties

What Task Are We Doing?

Chemical properties of wine → ML Model → Predicted Wine Class

The three models used are **Decision Tree, Random Forest, and KNN.**

Overall Flow to Remember

**Dataset → EDA → Visualization → Preprocessing → Model Building → Prediction
→ Evaluation → Model Comparison**

Final comparison is:

Decision Tree vs Random Forest vs KNN → Which model performs best?

1. Traditional / Manual Approach

In a manual approach, a human expert **examines the chemical properties and creates rules** to classify the wine.

For example:

If (Alcohol > 13 AND Flavanoids > 2.5) → Class 1

Another rule might be:

If (Alcohol < 12.5 AND Proline < 700) → Class 2

The problem is that with **13 chemical features**, creating accurate rules manually becomes difficult. The relationships between the features may also be complex.

2. Machine Learning Approach

- In ML, **we don't manually define these rules**. Instead, we provide the model with **178 wine samples** containing:

13 chemical features + their known wine class

- The ML algorithm examines these examples and **automatically discovers patterns and relationships** between the chemical properties and the three wine classes.

We don't explicitly tell the model: "Use this value as the boundary."

- The **algorithm learns suitable boundaries/patterns from the training data**.

3. When a New Wine Is Given

- Suppose we give the trained model the chemical properties of a new wine:

Alcohol + Malic Acid + Flavanoids + Magnesium + ... + Proline

- The model uses the **patterns it learned during training** and predicts: **This wine belongs to Class 1.**

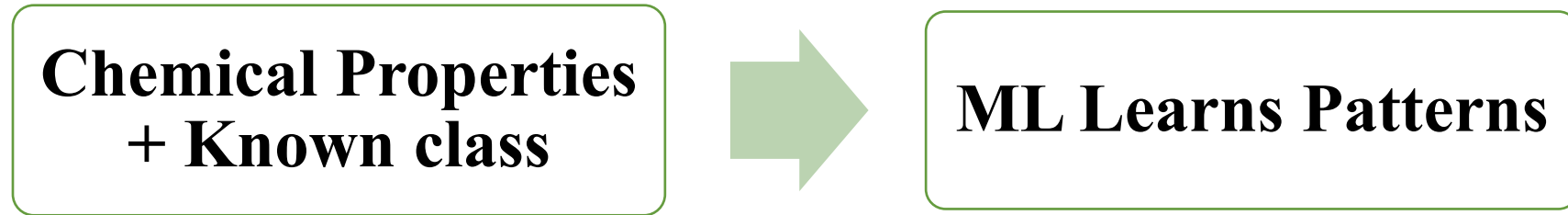
Training:

Chemical Properties + Known Class → ML learns patterns

Prediction:

New Chemical Properties → Learned Model → Predicted Class

ML Model Training



ML Model Prediction



2 Machine Learning Approach



In ML, we don't manually define rules.

We provide data, and the algorithm learns patterns automatically!



1. Provide Training Data

We provide the model with
178 wine samples
containing:



13 chemical features
(Input / X)



Known wine class (Target / y)
3 classes

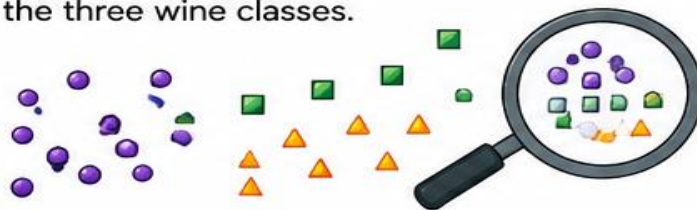
What does one sample look like?

Alcohol	Flavanoids	Color Intensity	Proline	...	Class
13.2	2.35	5.20	620	...	Class 1

Total: 178 samples (rows) × 13 features + 1 target

2. ML Learns Patterns

The ML algorithm examines all the examples and automatically
discovers patterns and relationships
between the chemical properties and the three wine classes.



Learns From Data – Not Rules

- We don't explicitly tell the model: "Use this value as the boundary."
- The algorithm learns suitable boundaries/patterns from the training data.

3. Learns Useful Combinations

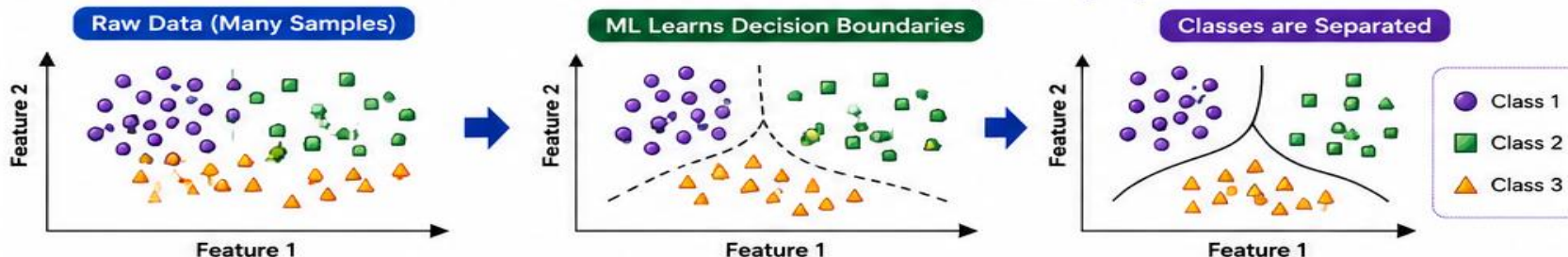
The model may learn that a
combination of features
(e.g., Alcohol, Flavanoids, Color Intensity, Proline, etc.)
is useful for distinguishing one class from another.

- Alc** Alcohol
- Flv** Flavanoids
- CI** Color Intensity
- Pro** Proline
- ...



ML finds the best way to separate the three classes.

What the Model Learns (Example)



4. Predict a New Wine

When a new wine is given
(with 13 chemical properties),
the model uses the learned
patterns to predict its class.



New Wine
(13 Features)



Trained
Model



Predicted
Wine Class
(Class 1 / 2 / 3)



Key Takeaway:

We provide **examples** (chemical properties + known class) → **ML learns patterns** → Can **accurately classify** new wines!

Thank you ...